

Required Readings

Required readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Zhang, Ashton, et al. *Dive into Deep Learning*. <https://d2l.ai/d2l-en.pdf>.

- **Required pages:** Chapter 5: Sections 5.4, 5.5, 5.6, and 5.7 (practice on Kaggle)
- **Estimated reading time:** 1 hour

Module 1: Lesson 1

Multilayer Perceptrons (MLPs)



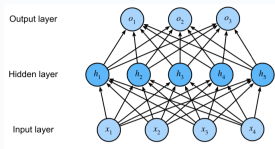
Outline

- ▶ Revisiting MLPs
- ▶ Initialization: vanishing and exploding gradients
- ▶ A quick note on regularization
- ▶ Dropout

Revisiting MLPs

During the last module of the Machine Learning course, we introduced a simple example of a deep Neural Network (NN): Multilayer Perceptrons (MLPs). This is precisely the point of departure for the Deep Learning for Finance course.

- Remember that we can express an MLP as a series of hidden layers with different units (neurons), each of which is activated (or not) given an activation function that incorporates the non-linearity into the process:



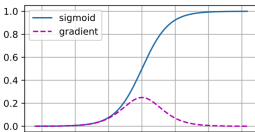
While we already know the basics about how these NNs work (e.g., forward and backprop), there are still many features to discuss that can help increase the efficiency and performance of our models. This is what we'll start doing in this lesson.

This lesson is based on Chapter 5 of Zhang et al.'s book *Dive into Deep Learning*.

Parameter initialization

Let's start discussing these by looking at the first task we need to perform when training: parameter initialization.

- ▶ So far, we have given relatively low importance to the way parameters (W and b) were initialized.
- ▶ We simply said that this was better done by selecting random numbers from a pre-specified distribution (e.g., uniform).
- ▶ However, the way that we initialize these parameters (e.g., the distribution we choose) can have significant effects on training.
- ▶ One of the most perilous effects of initialization choices is related to the (in)stability of gradients. The random choice of weights can lead to updates in parameters while training that are either:
 - Too large → Exploding gradient problem → difficult for gradient descent to converge
 - Too small → Vanishing gradient problem → difficult for algorithm to learn



How to initialize parameters

Is there any way to get rid of problems related to exploding/vanishing gradients?

Sadly, there is no universal way to get rid of the problems related to parameter initialization. There are, however, some things that we can do in order to mitigate their impact:

► **Default initialization in Keras:** One lazy option is to trust that Keras is taking care of the problem for us:

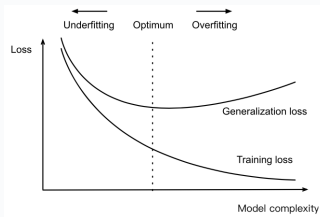
- Indeed, Keras has a default option for dense layers (the only ones we have seen so far) that relies on a uniform distribution called glorot uniform. Here, you have the link to the different default options when we set a dense layer: https://keras.io/api/layers/core_layers/dense/
- Here, you have the link to the Keras documentation for all the different initializers: <https://keras.io/api/layers/initializers/>

► **Xavier initialization:**

- Probably the one most commonly used in practice (actually glorot uniform is a type of Xavier)
- The typical Xavier initialization uses random weights from a Gaussian (or uniform) distribution with zero mean but conditioning the variance to avoid exploding/vanishing problems.

Regularization

Machine learning models, as we've already learned, are always subject to the trade-off between in- and out-of-sample performance. Deep learning models based on neural networks are not an exception to this problem:



When our models did overfit, we already worked out some solutions based on **regularization** of the loss function. These are also applicable in deep learning models, as we will see:

- ▶ Lasso (ℓ_1 -norm)
- ▶ Ridge (ℓ_2 -norm)
- ▶ Early Stopping

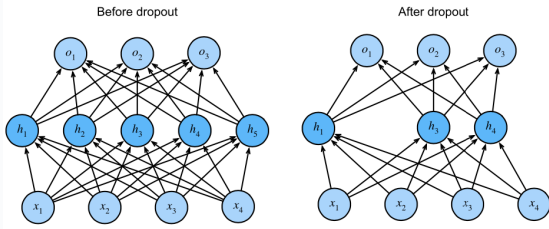
Dropout

A common form of regularization in deep learning that we have not yet talked about is **Dropout**.

Regularization norms are, in essence, ways to aim for a simpler model, which makes sense in light of the previous graph.

Dropout is a little different because it basically consists of adding noise to the model:

- ▶ It is called **dropout** because we literally drop out some neurons during model training.
- ▶ On each iteration during model training, we will zero-out (de-activate) a fraction of units in each layer.
- ▶ Typically, we will not use dropout in the test sample.



Summary of Lesson 1

In Lesson 1, we have:

- ▶ Revisited MLPs
- ▶ Discussed different options for parameter initialization
- ▶ Introduced dropout

⇒ **References for this lesson:**

Chapter 5, Sections 5.4 to 5.7 of [Zhang, Ashton, et al. *Dive into Deep Learning*](#).

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement all these novel things in practice in an increasingly deeper neural network based on MLPs.

Module 1: Lesson 2

MLP for classification: Timing AAPL stock



Outline

- ▶ MLP for classification
- ▶ Binary cross-entropy loss function
- ▶ Class weights
- ▶ Model predictive performance
- ▶ Trading strategy performance

MLP for classification

In the last module of Machine Learning, we dealt with an example of how to using neural networks, specifically MLPs, to time the momentum factor. Arguably, we used a somewhat simple model that did not really deliver an stellar performance when it came to predicting the forward-looking return of the factor.

Now, we'll insist on this challenging task with some changes:

- ▶ Perform a **classification** task: instead of focusing on predicting the forward-looking return of a financial series, we will merely try to predict whether this return will be positive ($y_i = 1$) or negative ($y_i = 0$).
- ▶ Focus on a **single stock (AAPL)**. Rather than looking at a factor that explains the cross-section of returns, we will limit our investigation to a single stock time series.
- ▶ Incorporate new features such as **dropout** and **class weights**.

Before jumping to a road map on the implementation you will see in detail in the notebook, a few caveats:

1. Focus on a single stock (AAPL). The model may not work as well/badly for others (feel free to try!)
2. Focus on a specific time period (1980–2022). The model may not work as well/badly for others (or future!).

Building our model (1/2)

We'll build our model in a very similar way to what we have done before:

- ▶ **Layers:** 3 hidden layers with 25, 15, and 10 units respectively + output layer
- ▶ **Dropout:** dropout layer of 20% after each hidden layer
- ▶ **Optimizer & Learning rate:** Adam & 10^{-6}
- ▶ **Activation function:** ReLU for all hidden layers + Sigmoid for output layer

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 25)	150
dropout (Dropout)	(None, 25)	0
dense_1 (Dense)	(None, 15)	390
dropout_1 (Dropout)	(None, 15)	0
dense_2 (Dense)	(None, 10)	160
dropout_2 (Dropout)	(None, 10)	0
dense_3 (Dense)	(None, 1)	11
=====		
Total params: 711		
Trainable params: 711		
Non-trainable params: 0		

Building our model (2/2)

More (new!) things about our model:

- **Loss function:** Binary cross-entropy

$$H(y_i) = -\frac{1}{N} \sum_{i=1}^N y_i \log(p(y_i)) + (1 - y_i) \log(1 - p(y_i))$$

- **Metric:** (binary) accuracy to evaluate our model performance

```
import tensorflow as tf

tf.keras.backend.clear_session()
tf.random.set_seed(1234)

act_fun = 'relu'
hp_units = 25
hp_units_2 = 15
hp_units_3 = 10
n_dropout = 0.2

model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Dense(units = hp_units, activation=act_fun))
model.add( tf.keras.layers.Dropout(n_dropout))
model.add(tf.keras.layers.Dense(units = hp_units_2, activation=act_fun))
model.add( tf.keras.layers.Dropout(n_dropout))
model.add(tf.keras.layers.Dense(units = hp_units_3, activation=act_fun))
model.add( tf.keras.layers.Dropout(n_dropout))
model.add(tf.keras.layers.Dense(units = 1, activation='sigmoid'))

hp_lr = 1e-5

adam = tf.keras.optimizers.Adam(lr=hp_lr)

model.compile(optimizer = adam, loss='binary_crossentropy', metrics=['accuracy'])
```

Class weights

One problem that may always be present when we perform classification tasks is the under-representation of a certain class in the sample data. This can cause the model to overlook an important part of the data in the sample while training.

- ▶ For example, suppose we want to predict firm bankruptcy using imperfect historical data. Bankruptcy may not be a very frequent event in our sample but is truly our main variable of interest.
- ▶ To mitigate this problem, we can make the model pay more attention to certain classes. This can be done by introducing **class weights**, which simply consist of directing Keras to pay more attention to a certain class.
- ▶ In our case, we will try to overweight the negative classes in our sample (i.e., those instances where the 120-day-forward return on AAPL is negative).
- ▶ This is actually something we will introduce while training the model, similar to the EarlyStopping callback:

```
from sklearn.utils import class_weight

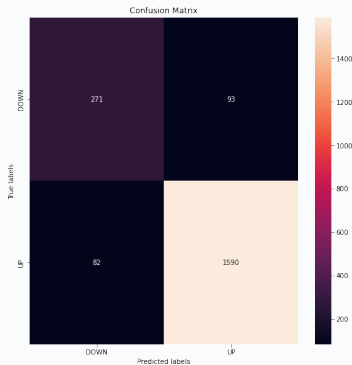
class_weight = {0: (np.mean(y_train)/0.5)*1.2, 1: 1.0}
print(class_weight)

history = model.fit(X_train, y_train, validation_split=0.2, epochs=500, batch_size=32,
                    verbose=2, callbacks=[es], class_weight=class_weight)
```

Model predictive performance

So how does our predictive model perform in the test sample?

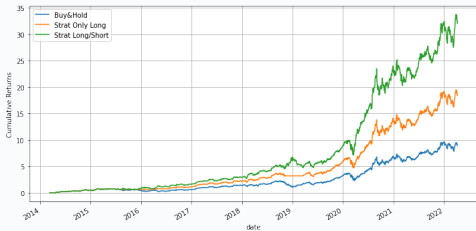
- To assess that, we will come back to an old friend: the confusion matrix!



- Model accuracy in Test: 91.40%

Trading strategy performance

Predictive power of the model in a test sample is important to assess the performance of our model. But if we want to investigate whether we could use this for trading, we better perform a backtest:



Usual **caveats** on backtesting:

- ▶ Specific stock (AAPL)
- ▶ Specific time period for training and testing (1980–2022)
- ▶ Based on historical information (what about things that never happened before?)

Summary of Lesson 2

In Lesson 2, we have:

- ▶ Developed an MLP model for classification to predict whether a certain stock will deliver a positive or negative return in the next few days
- ▶ Enhanced the performance of our MLP neural networks with features such as dropout or class weights.

⇒ **TO DO NEXT:** Go to the associated **Jupyter Notebook** to cover the full content of this lesson.

⇒ **In the next lesson**, we'll keep improving our neural network models by introducing hyperparameter tuning.

DEEP LEARNING FOR FINANCE

MODULE 1 | LESSON 2

Multilayer Perceptron: Market Timing in AAPL Stock

Reading Time	60 minutes
Prior Knowledge	MLP, Market timing, Python
Keywords	MLP, Classification, Neural Network, Dropout

*In this lesson we will start increasing the complexity of our Neural Networks by considering more dense MLPs that incorporate features such as **Dropout** in order to enhance its performance. Also, different from the regression example we used in the Machine Learning course on timing the Momentum factor, we will deal here with a **classification** problem. Specifically, we will try to design a strategy that times the return of Apple stock ('AAPL'). For that, we will employ an MLP network that aims to predict whether the future return of AAPL is positive or negative. Based on the predictions from the model, we will later on check how our strategy performs.*

1. Data and sources

As with any other Deep Learning endeavor, the single most important and necessary ingredient is data. Different from what we did in the Momentum timing example, in this case we will retrieve Apple stock price data from Yahoo! Finance using the famous **yfinance** library in Python, which you are already familiar with:

Darell Moodley inserted this code

```
In [1]: # !pip install keras-tuner
import tensorflow as tf
from tensorflow import keras
import keras_tuner as kt
import numpy as np
```

```
2025-09-24 07:55:33.743440: I tensorflow/core/util/port.cc:153] oneDNN
custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-09-24 07:55:33.796784: I
tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is
optimized to use available CPU instructions in performance-critical
operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16
AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
2025-09-24 07:55:35.377415: I tensorflow/core/util/port.cc:153] oneDNN
custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

```
In [2]: n_data = 5000

# Create a simple dataset for demonstration
# We'll use a regression problem, where  $y = 2x + 1$  with some noise.
X = np.random.rand(n_data, 1) * 10
y = 2 * X + 1 + np.random.randn(n_data, 1) * 2

# Split data into training and validation sets
from sklearn.model_selection import train_test_split
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
random_state=42)
```

```
In [3]: def build_model(hp):
        model = keras.Sequential()

        # Input layer
        model.add(keras.layers.Input(shape=(1,)))

        # Hyperparameter search for the number of hidden layers

        for i in range(hp.Int('num_layers', min_value=5, max_value=60,
step=5)):
            model.add(tf.keras.layers.Dense(units=15, activation='relu'))

        # Output layer
        model.add(keras.layers.Dense(units=1))

        # Compile the model
        model.compile(
            optimizer='adam',
            loss='mean_squared_error',
            metrics=['mean_absolute_error'])

        return model
```

```
In [ ]: ### correct answer
        for i in range(hp.Int('num_layers', min_value=5, max_value=60, step=5)):
            model.add(tf.keras.layers.Dense(units=15, activation='relu'))
```

```
In [4]: tuner = kt.RandomSearch(
        hypermodel=build_model,
        objective='val_mean_absolute_error',
        max_trials=5, # The number of models to test
        executions_per_trial=2, # Number of times to run each model to reduce
        variance
        directory='my_dir',
        project_name='tuning_hidden_layers')

# # Run the search
tuner.search(X_train, y_train, epochs=20, validation_data=(X_val, y_val))

# # Get the optimal hyperparameters
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]
print(f"The optimal number of hidden layers is:
{best_hps.get('num_layers')}")

# # Get the best model
best_model = tuner.get_best_models(num_models=1)[0]
best_model.summary()
```

Reloading Tuner from my_dir/tuning_hidden_layers/tuner0.json

The optimal number of hidden layers is: 45

/usr/local/lib/python3.11/site-packages/keras/src/saving/saving_lib.py:757:
UserWarning: Skipping variable loading for optimizer 'adam', because it has
2 variables whereas the saved optimizer has 186 variables.
saveable.load_own_variables(weights_store.get(inner_path))

Model: "sequential"

Layer (type) Param #	Output Shape	
dense (Dense) 30	(None, 15)	
dense_1 (Dense) 240	(None, 15)	
dense_2 (Dense) 240	(None, 15)	
dense_3 (Dense) 240	(None, 15)	
dense_4 (Dense) 240	(None, 15)	
dense_5 (Dense) 240	(None, 15)	
dense_6 (Dense) 240	(None, 15)	
dense_7 (Dense) 240	(None, 15)	
dense_8 (Dense) 240	(None, 15)	
dense_9 (Dense) 240	(None, 15)	
dense_10 (Dense) 240	(None, 15)	
dense_11 (Dense) 240	(None, 15)	
dense_12 (Dense) 240	(None, 15)	
dense_13 (Dense) 240	(None, 15)	
dense_14 (Dense) 240	(None, 15)	

dense_15 (Dense)	(None, 15)	
240		
dense_16 (Dense)	(None, 15)	
240		
dense_17 (Dense)	(None, 15)	
240		
dense_18 (Dense)	(None, 15)	
240		
dense_19 (Dense)	(None, 15)	
240		
dense_20 (Dense)	(None, 15)	
240		
dense_21 (Dense)	(None, 15)	
240		
dense_22 (Dense)	(None, 15)	
240		
dense_23 (Dense)	(None, 15)	
240		
dense_24 (Dense)	(None, 15)	
240		
dense_25 (Dense)	(None, 15)	
240		
dense_26 (Dense)	(None, 15)	
240		
dense_27 (Dense)	(None, 15)	
240		
dense_28 (Dense)	(None, 15)	
240		
dense_29 (Dense)	(None, 15)	
240		
dense_30 (Dense)	(None, 15)	
240		

dense_31 (Dense)	(None, 15)	
240		
dense_32 (Dense)	(None, 15)	
240		
dense_33 (Dense)	(None, 15)	
240		
dense_34 (Dense)	(None, 15)	
240		
dense_35 (Dense)	(None, 15)	
240		
dense_36 (Dense)	(None, 15)	
240		
dense_37 (Dense)	(None, 15)	
240		
dense_38 (Dense)	(None, 15)	
240		
dense_39 (Dense)	(None, 15)	
240		
dense_40 (Dense)	(None, 15)	
240		
dense_41 (Dense)	(None, 15)	
240		
dense_42 (Dense)	(None, 15)	
240		
dense_43 (Dense)	(None, 15)	
240		
dense_44 (Dense)	(None, 15)	
240		
dense_45 (Dense)	(None, 1)	
16		

Total params: 10,606 (41.43 KB)

Trainable params: 10,606 (41.43 KB)

Non-trainable params: 0 (0.00 B)

In []:

In []:

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [2]: # Read the CSV file
file_path = 'AAPL.csv'
df = pd.read_csv(file_path, delimiter=';', decimal=',', parse_dates=
['Date'], date_format='%m/%d/%y')
df = df.sort_values('Date')
df = df.set_index('Date')

df.head()
```

Out[2]:

	Price	CVol	Change	% Change	% Return	Total Return (Gross)	Cumulative Return %	Open	High	Low	Cumulative Change %
Date											
1980-12-12	0.13	46903360.0	0.13	NaN	NaN	0.13	NaN	NaN	0.13	0.13	NaN
1980-12-15	0.12	17588480.0	-0.01	-5.22	-5.22	0.12	-5,22	NaN	0.12	0.12	-5,22
1980-12-16	0.11	105728.0	-0.01	-7.32	-7.32	0.11	-12,16	NaN	0.11	0.11	-12,16
1980-12-17	0.12	8644160.0	0.00	2.48	2.48	0.12	-9,98	NaN	0.12	0.12	-9,98
1980-12-18	0.12	7344960.0	0.00	2.86	2.86	0.12	-7,40	NaN	0.12	0.12	-7,40

```
In [3]: # Rename '% Return' column to 'Ret'
df = df.rename(columns={'% Return': 'Ret'})

# Define the name variable
name = 'Ret'

# Set 'Date' as index and keep only 'Ret' column
df = df.dropna()

df.head()
```

Out[3]:

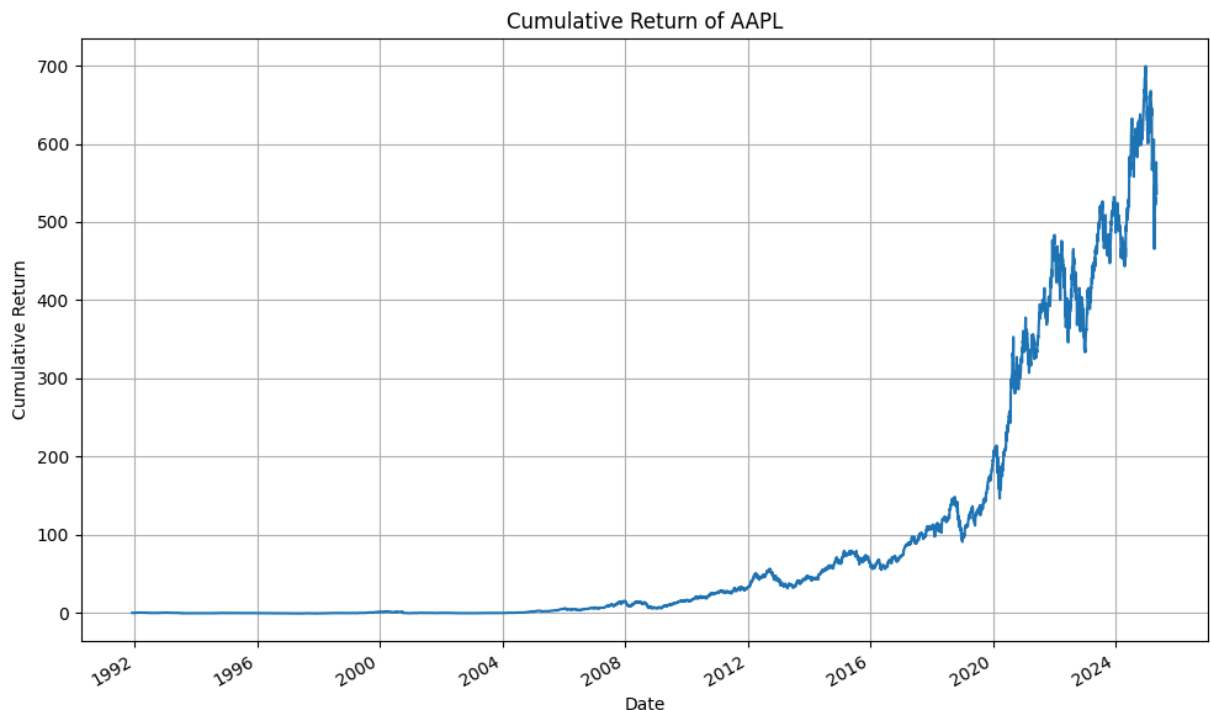
	Price	CVol	Change	% Change	Ret	Total Return (Gross)	Cumulative Return %	Open	High	Low	Cumulative Change %
Date											
1991-12-02	0.46	118899320.0	0.01	1.97	1.97	0.48	276,80	0.45	0.46	0.45	260,13
1991-12-03	0.45	102860910.0	-0.01	-2.42	-2.42	0.47	267,70	0.46	0.46	0.45	251,44
1991-12-05	0.45	99198490.0	0.00	-0.99	-0.99	0.47	264,06	0.45	0.46	0.44	247,96
1991-12-06	0.44	196985780.0	-0.01	-2.50	-2.50	0.46	254,96	0.44	0.44	0.43	239,26
1991-12-09	0.44	97832080.0	0.00	0.77	0.77	0.46	257,69	0.44	0.45	0.44	241,87

We will retrieve daily stock price data from AAPL (Apple Inc.) from January 1st, 1980, but we keep only data that do not have any missing observations, reducing the sample size substantially.

Once we have daily prices, we can compute daily returns, which will be used to construct the inputs (and output) of our MLP:

```
In [4]: # Calculate cumulative return
df['Cumulative Return'] = (1 + df['Ret'] / 100).cumprod() - 1

# Plot the cumulative return
plt.figure(figsize=(10, 6))
df['Cumulative Return'].plot()
plt.title('Cumulative Return of AAPL')
plt.xlabel('Date')
plt.ylabel('Cumulative Return')
plt.grid(True)
plt.tight_layout()
plt.show()
```



2. Timing Apple stock with Multilayer Perceptron (MLPs)

As we have already mentioned, the purpose of our MLP is to predict the future return (we will shortly see for which time horizon and how) of AAPL using past returns. Hence, one of the first things we must do is decide and define the inputs and outputs of the model.

2.1 Inputs and outputs

As you already know, there are multiple choices to make when selecting the inputs of a MLP model. Some of them are based on sound theory, but also some simply stem from a trial and error process. In this case, we will apply a very similar approach to what we did when building our models for factor momentum timing and consider as inputs in the network the stock returns from the past 25, 60, 90, 120 and 240 days. Once again, after going over and understanding the entire notebook here, please feel free to play around with these time frames or even add more/less inputs into the MLP to see how its performance changes.

```
In [5]: df = df[['Ret']]

df["Ret25_i"] = df[name].rolling(25).apply(lambda x: 100 * (np.prod(1 + x / 100) - 1))
df["Ret60_i"] = df[name].rolling(60).apply(lambda x: 100 * (np.prod(1 + x / 100) - 1))
df["Ret90_i"] = df[name].rolling(90).apply(lambda x: 100 * (np.prod(1 + x / 100) - 1))
df["Ret120_i"] = df[name].rolling(120).apply(lambda x: 100 * (np.prod(1 + x / 100) - 1))
df["Ret240_i"] = df[name].rolling(240).apply(lambda x: 100 * (np.prod(1 + x / 100) - 1))

df = df.dropna()
df.tail(10)
```

```
Out[5]:
```

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i
Date						
2025-04-23	2.43	-3.795072	-10.881083	-17.324816	-11.375229	13.166308
2025-04-24	1.84	-3.186662	-12.437332	-15.363482	-10.514112	14.812281
2025-04-25	0.44	-2.242569	-12.454764	-15.498093	-10.228100	15.098768
2025-04-28	0.41	-3.719238	-11.440489	-15.210988	-8.459465	14.426408
2025-04-29	0.51	-4.309509	-10.388438	-15.764123	-6.287033	15.809066
2025-04-30	0.61	-5.026928	-6.678198	-16.064459	-4.444495	14.500296
2025-05-01	0.39	-3.703195	-8.241178	-13.894452	-3.686575	14.238568
2025-05-02	-3.74	-8.267882	-11.549127	-17.690963	-7.887429	8.640630
2025-05-05	-3.15	-8.729653	-14.608582	-21.754709	-10.493604	5.155357
2025-05-06	-0.19	-10.636714	-12.675026	-22.144726	-12.535408	4.934575

Defining the output: Classification

Finally, we need to define our output label/s. We have decided to focus on the +120 (trading) days return for Apple stock. Remember that in this case we will perform a classification task with our MLP model, so that we simply aim to predict whether, on a given time t , the return of AAPL from t to $t + 120$ days will be positive or negative (note how a zero return, although unlikely, will also be classified as negative).

Therefore, we first investigate, at a given time t , what would be the 120-day return on AAPL. Then, we calculate our output variable, keeping in mind that we

will be running a classification task and, hence, we need to convert our output variable to a 0, 1 variable (0 for negative 120 days return, 1 for positive):

```
In [6]: df["Ret120"] = df["Ret120_i"].shift(-120)
df["Output"] = df["Ret120"] > 0
df["Output"] = df["Output"].astype(int)
del df["Ret120"]
df = df.dropna()
df.tail(10)
```

```
Out[6]:
```

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i	Output
Date							
2025-04-23	2.43	-3.795072	-10.881083	-17.324816	-11.375229	13.166308	0
2025-04-24	1.84	-3.186662	-12.437332	-15.363482	-10.514112	14.812281	0
2025-04-25	0.44	-2.242569	-12.454764	-15.498093	-10.228100	15.098768	0
2025-04-28	0.41	-3.719238	-11.440489	-15.210988	-8.459465	14.426408	0
2025-04-29	0.51	-4.309509	-10.388438	-15.764123	-6.287033	15.809066	0
2025-04-30	0.61	-5.026928	-6.678198	-16.064459	-4.444495	14.500296	0
2025-05-01	0.39	-3.703195	-8.241178	-13.894452	-3.686575	14.238568	0
2025-05-02	-3.74	-8.267882	-11.549127	-17.690963	-7.887429	8.640630	0
2025-05-05	-3.15	-8.729653	-14.608582	-21.754709	-10.493604	5.155357	0
2025-05-06	-0.19	-10.636714	-12.675026	-22.144726	-12.535408	4.934575	0

It is always useful to see some summary statistics of the different variables in our model. We can very easily observe them with the '**.describe()**' feature in Python:

```
In [7]: df.describe()
```

Out[7]:

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i	Output
count	8035.00 0000	8035.00 0000	8035.00 0000	8035.00 0000	8035.00 0000	8035.00 0000	8035.00 0000
mean	0.11376 9	2.87834 7	7.26045 2	11.1430 45	15.1130 84	32.5692 85	0.69159 9
std	2.69981 6	13.3948 73	21.6019 84	27.5691 07	33.1061 80	56.0242 48	0.46186 2
min	-51.8700 00	-69.5981 32	-77.0749 30	-72.4838 02	-74.7292 50	-75.0073 29	0.00000 0
25%	-1.16000 0	-4.86031 1	-5.00199 8	-5.04251 2	-4.13941 2	-6.00292 9	0.00000 0
50%	0.10000 0	2.83195 7	7.05025 0	10.6256 07	14.6418 36	29.0478 10	1.00000 0
75%	1.37500 0	10.5473 27	19.2463 92	26.0214 18	30.6645 92	59.0455 64	1.00000 0
max	33.2300 00	121.286 374	116.442 841	135.727 260	167.513 049	300.683 525	1.00000 0

2.2 Train-Test samples and Scaling

Next important step in building our model is in defining our train and test samples, together with the scaling of variables. As we have usually done before, we will take 20% of observations and devote them to testing, while 80% will be used for training the model. Please note, once again, the importance of doing this in chronological order!

```
In [8]: # Train-test split
ts = int(0.2 * len(df)) # Number of observations in test sample (20%)
split_time = len(df) - ts # Start of test sample
test_time = df.index[split_time:] # Extract Date index for test period
Ret_vector = df['Ret'][split_time:].values # Extract Ret values for test period
df.tail()
```

Out[8]:

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i	Output
Date							
2025-04-30	0.61	-5.02692 8	-6.67819 8	-16.0644 59	-4.44449 5	14.5002 96	0
2025-05-01	0.39	-3.70319 5	-8.24117 8	-13.8944 52	-3.68657 5	14.2385 68	0
2025-05-02	-3.74	-8.26788 2	-11.5491 27	-17.6909 63	-7.88742 9	8.64063 0	0
2025-05-05	-3.15	-8.72965 3	-14.6085 82	-21.7547 09	-10.4936 04	5.15535 7	0
2025-05-06	-0.19	-10.6367 14	-12.6750 26	-22.1447 26	-12.5354 08	4.93457 5	0

Now we are ready to use **sklearn** to formally define the input and output matrices for training (**X_train** and **y_train**) and test (**X_test** and **y_test**). One more time, make sure you keep the 'shuffle' option set to 'False'!

```
In [9]: from sklearn.model_selection import train_test_split

Xdf, ydf = df.iloc[:, 1:-1], df.iloc[:, -1]
X = Xdf.astype("float32")
y = ydf.astype("float32")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=ts, shuffle=False
) # It is important to keep "shuffle=False"
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(6428, 5) (1607, 5) (6428,) (1607,)
```

The last 'print' in the previous chunk of code gives as an output very useful information specially for a later time where we will move on to more complex models. In this case, we know that we have 6,428 observations in training for 5 different input series. Just as a concept check, could you say how many observations are in the test sample?

- **What about scaling?**

As you see, we have not implemented scaling here. Scaling of variables is actually a very good practice that we recommend you always follow. In this case, since we are working with returns from the same stock, the impact that scaling of inputs will have is presumably very limited. (To be clear, we use 'scaled' data by taking prices that adjust for any dividends and stock splits). Nonetheless, we leave this task to you. Once you have completed the whole notebook, come back here and scale input variables to see whether model performance changes by a lot. Feel free to go back to the last module of Machine Learning, where we introduced scaling.

2.3 Model and Training

Finally, it is time to set up our model. In this case, we will use 3 hidden layers with 25, 15 and 10 units respectively, and a final single-unit output layer. For all the hidden layers, we will use a **ReLU** activation function, whereas we will opt for a **sigmoid** activation in the output layer.

- **Dropout**

Novel to MLP models we have used before (in the Machine Learning course), we are here introducing a **dropout layer** after each of the hidden layers. As you already know, dropout randomly sets some units of a hidden layer to zero. The question of how many units (i.e., frequency rate) is up to the user to define. In this case, we set **n_dropout = 0.2** to shut down 20% of the units in the layer. You can find more info on how the dropout layers work in the official Keras documentation: https://keras.io/api/layers/regularization_layers/dropout/

- **Loss function?**

So far, we have dealt with regression problems where we used loss functions based on Mean Squared Error (MSE) or Mean Absolute Error (MAE). But now we are in a classification problem with 2 labels (0, 1), so we select a loss function, **binary cross-entropy** that is essentially a log-likelihood:

$$H(y_i) = -\frac{1}{N} \sum_{i=1}^N y_i \log(p(y_i)) + (1 - y_i) \log(1 - p(y_i))$$

For more information on how this loss function works, check the **official documentation** in Keras:

https://www.tensorflow.org/api_docs/python/tf/keras/losses/BinaryCrossentropy

A simple explanation of binary cross-entropy may be found here:

<https://towardsdatascience.com/understanding-binary-cross-entropy-log-loss-a-visual-explanation-a3ac6025181a>

- **Metric for loss function?**

Still, one question remains about which metric would we use for our loss function based on binary cross-entropy. A **metric** is simply a function that is used to judge the performance of your model. See all metrics available in Keras here:

<https://keras.io/api/metrics/>

Here, we choose to judge our model performance using '**accuracy**', which in this case corresponds to binary accuracy:

https://keras.io/api/metrics/accuracy_metrics/#binaryaccuracy-class

```
In [10]: import tensorflow as tf

tf.keras.backend.clear_session() # We clear the backend to reset the
random seed process
tf.random.set_seed(
    1234
) # We will set a random seed so that results obtained are somewhat
replicable

act_fun = "relu"
hp_units = 25
hp_units_2 = 15
hp_units_3 = 10
n_dropout = 0.2

model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Dense(units=hp_units, activation=act_fun))
model.add(tf.keras.layers.Dropout(n_dropout))
model.add(tf.keras.layers.Dense(units=hp_units_2, activation=act_fun))
model.add(tf.keras.layers.Dropout(n_dropout))
model.add(tf.keras.layers.Dense(units=hp_units_3, activation=act_fun))
model.add(tf.keras.layers.Dropout(n_dropout))
model.add(tf.keras.layers.Dense(units=1, activation="sigmoid"))

hp_lr = 1e-5 # Learning rate

adam = tf.keras.optimizers.Adam(learning_rate=hp_lr) # Adam optimizer

model.compile(optimizer=adam, loss="binary_crossentropy", metrics=
["accuracy"])
```

```
2025-05-07 07:31:19.938792: I tensorflow/core/util/port.cc:153] oneDNN
custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-05-07 07:31:19.983019: I
tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is
optimized to use available CPU instructions in performance-critical
operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16
AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
2025-05-07 07:31:21.038369: I tensorflow/core/util/port.cc:153] oneDNN
custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

2.3.1 Validation and callbacks (Earlystopping)

We have already built our model, but there are still a few questions that need an answer when training it: how many epochs do we train for? when does the model stop?

To solve these questions, we will use **EarlyStopping**, a method you are already familiar with (feel free to revisit the documentation for Module 7 of Machine Learning). In this case the variable that we will monitor will be model accuracy in the validation set. Naturally, we will aim to obtain the maximum accuracy in

validation, and set the patience to 20, so that model training will stop after 20 epochs in which there is no improvement in the accuracy that the model weights yield in the validation set.

```
In [11]: es = tf.keras.callbacks.EarlyStopping(  
    monitor="val_accuracy",  
    mode="max",  
    verbose=1,  
    patience=20,  
    restore_best_weights=True,  
)
```

2.3.2 Classification on imbalance data: class_weight

When we work on classification tasks, there is always the possibility that one of the labels we are trying to predict is underrepresented in the training sample. Ideally you would want the model to give a heavier weight to underrepresented labels so that you do not overlook this in future prediction. For example, suppose that you are trying to predict corporate default, but your data has very small percentage of observation when a firm actually defaults. You may want the model to pay more attention to these observations where the actual action occurs. For that, we can use **class_weights**.

Class-weights essentially consist on passing Keras a weight for each class in the sample, so that we can make the model focus on a particular class more than it will based on its representation in the sample.

Here you have a complete tutorial on how to perform these kind of tasks that use imbalance data in Keras. Please note that his tutorial covers a wide range of topics that we will eventually cover as well. In order not to get lost in details that are not important for now, we suggest you focus on the 'Class weights' section: https://www.tensorflow.org/tutorials/structured_data/imbalanced_data?hl=en

In this case, we would like the model to pay more attention to labels of 0, which correspond to days in which the 120-day-ahead return is negative. There are many ways to assign and modify class weights, we follow here a simple one, completely discretionary:

```
In [12]: class_weight = {0: (np.mean(y_train) / 0.5) * 1.2, 1: 1.0}  
print(class_weight)  
{0: 1.6312382698059082, 1: 1.0}
```

Finally, we are ready to train our model with all these ingredients!

```
In [13]: history = model.fit(  
    X_train,  
    y_train,  
    validation_split=0.2,  
    epochs=500,  
    batch_size=32,  
    verbose=2,  
    callbacks=[es],  
    class_weight=class_weight,  
)
```

Epoch 1/500
161/161 - 1s - 9ms/step - accuracy: 0.5998 - loss: 5.6416 - val_accuracy: 0.7706 - val_loss: 1.2708
Epoch 2/500
161/161 - 0s - 3ms/step - accuracy: 0.5862 - loss: 5.3652 - val_accuracy: 0.7714 - val_loss: 1.1826
Epoch 3/500
161/161 - 1s - 4ms/step - accuracy: 0.5994 - loss: 5.3257 - val_accuracy: 0.7706 - val_loss: 1.0847
Epoch 4/500
161/161 - 0s - 2ms/step - accuracy: 0.5838 - loss: 5.1315 - val_accuracy: 0.7714 - val_loss: 0.9956
Epoch 5/500
161/161 - 0s - 2ms/step - accuracy: 0.5957 - loss: 4.7177 - val_accuracy: 0.7722 - val_loss: 0.9188
Epoch 6/500
161/161 - 0s - 2ms/step - accuracy: 0.5867 - loss: 4.6330 - val_accuracy: 0.7714 - val_loss: 0.8548
Epoch 7/500
161/161 - 0s - 2ms/step - accuracy: 0.5932 - loss: 4.4603 - val_accuracy: 0.7706 - val_loss: 0.7948
Epoch 8/500
161/161 - 0s - 2ms/step - accuracy: 0.5685 - loss: 4.3878 - val_accuracy: 0.7698 - val_loss: 0.7364
Epoch 9/500
161/161 - 0s - 2ms/step - accuracy: 0.5762 - loss: 4.1500 - val_accuracy: 0.7621 - val_loss: 0.6938
Epoch 10/500
161/161 - 0s - 2ms/step - accuracy: 0.5694 - loss: 4.1567 - val_accuracy: 0.7605 - val_loss: 0.6574
Epoch 11/500
161/161 - 0s - 2ms/step - accuracy: 0.5729 - loss: 3.8353 - val_accuracy: 0.7543 - val_loss: 0.6245
Epoch 12/500
161/161 - 0s - 2ms/step - accuracy: 0.5681 - loss: 4.0955 - val_accuracy: 0.7465 - val_loss: 0.6002
Epoch 13/500
161/161 - 1s - 4ms/step - accuracy: 0.5722 - loss: 3.6306 - val_accuracy: 0.7403 - val_loss: 0.5881
Epoch 14/500
161/161 - 0s - 2ms/step - accuracy: 0.5712 - loss: 3.5619 - val_accuracy: 0.7278 - val_loss: 0.5813
Epoch 15/500
161/161 - 0s - 2ms/step - accuracy: 0.5710 - loss: 3.3990 - val_accuracy: 0.7068 - val_loss: 0.5795
Epoch 16/500
161/161 - 1s - 4ms/step - accuracy: 0.5490 - loss: 3.3264 - val_accuracy: 0.6921 - val_loss: 0.5795
Epoch 17/500
161/161 - 0s - 2ms/step - accuracy: 0.5545 - loss: 3.4711 - val_accuracy: 0.6820 - val_loss: 0.5798
Epoch 18/500
161/161 - 0s - 2ms/step - accuracy: 0.5537 - loss: 3.2934 - val_accuracy: 0.6750 - val_loss: 0.5800
Epoch 19/500
161/161 - 0s - 2ms/step - accuracy: 0.5580 - loss: 3.2438 - val_accuracy: 0.6625 - val_loss: 0.5826
Epoch 20/500
161/161 - 0s - 2ms/step - accuracy: 0.5537 - loss: 3.0105 - val_accuracy: 0.6594 - val_loss: 0.5858

```

Epoch 21/500
161/161 - 0s - 2ms/step - accuracy: 0.5371 - loss: 3.0470 - val_accuracy:
0.6493 - val_loss: 0.5900
Epoch 22/500
161/161 - 0s - 2ms/step - accuracy: 0.5424 - loss: 2.9000 - val_accuracy:
0.6376 - val_loss: 0.5926
Epoch 23/500
161/161 - 0s - 2ms/step - accuracy: 0.5449 - loss: 3.0201 - val_accuracy:
0.6159 - val_loss: 0.5980
Epoch 24/500
161/161 - 0s - 2ms/step - accuracy: 0.5447 - loss: 2.8814 - val_accuracy:
0.6003 - val_loss: 0.6010
Epoch 25/500
161/161 - 0s - 2ms/step - accuracy: 0.5288 - loss: 2.8589 - val_accuracy:
0.5902 - val_loss: 0.6030
Epoch 25: early stopping
Restoring model weights from the end of the best epoch: 5.

```

As usual, let's also inspect the summary of the model layers, which in the presence of dropout makes much more sense to have clear in mind:

```
In [14]: model.summary()
```

Model: "sequential"

Layer (type) Param #	Output Shape	
dense (Dense) 150	(None, 25)	
dropout (Dropout) 0	(None, 25)	
dense_1 (Dense) 390	(None, 15)	
dropout_1 (Dropout) 0	(None, 15)	
dense_2 (Dense) 160	(None, 10)	
dropout_2 (Dropout) 0	(None, 10)	
dense_3 (Dense) 11	(None, 1)	

Total params: 2,135 (8.34 KB)

Trainable params: 711 (2.78 KB)

Non-trainable params: 0 (0.00 B)
Optimizer params: 1,424 (5.57 KB)

Note that the presence of the dropout layer does not interfere with the number of parameters in the model. This is because dropout neither adds nor reduces the number of model parameters (or units) as whole, as units are randomly shut down during training epochs, but of course the unit affected is not always the same!

This section concludes all relative to the construction and training of the MLP classification model. Now, as we have usually done, let's explore if we could use this model, which achieves a decent accuracy in the validation set, to build a trading strategy: All rights reserved WQU WorldQuant University QQQQ

3. Financial performance of the model

As we have done before, let's now evaluate the use of this model for a financial strategy. It is very important that we clearly understand that evaluating the **financial performance** is a completely separate thing from evaluating the **predictive performance** of the model. Obviously, the two tasks are interrelated, but they may offer completely different conclusions after a thorough analysis. We have already trained our model and assess its fit. Now, we will check if the predictions delivered in the test sample are valid for developing a trading strategy!

3.1 Model performance in test sample

We will start by obtaining the predictions of the model in the test sample and its accuracy. Remember, we are evaluating how well our model predicts, using past AAPL returns for 5 different windows, whether the next 120 days returns is positive or negative. The way we will operate is defining model prediction as a 1 (i.e., positive return) if the probability assigned by the model is higher than 0.5; and 0 (i.e., negative return) else.

```
In [15]: import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import metrics

y_prob = model.predict(X_test)
y_pred = np.where(y_prob > 0.50, 1, 0)

acc = model.evaluate(X_test, y_test)
print("Model accuracy in test: ", acc)
```

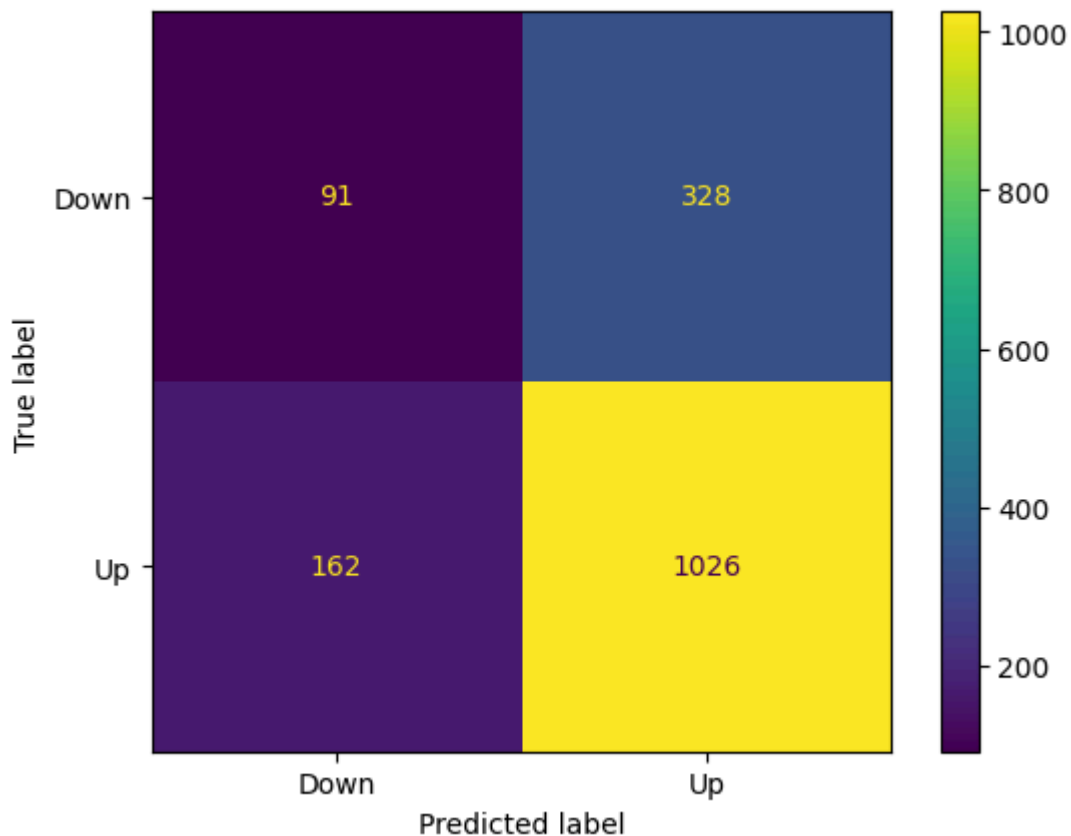
```
51/51 ————— 0s 1ms/step
51/51 ————— 0s 1ms/step - accuracy: 0.7859 - loss: 0.6321
Model accuracy in test: [0.9570235013961792, 0.6950840353965759]
```

As you can see, we obtain a very decent accuracy of the model in the test sample. But we would also like to see when does the model 'miss', and when it is mostly 'right' about a prediction. For that, we come to our old friend, the confusion matrix:

```
In [16]: ### DMoodley correction
from sklearn.metrics import ConfusionMatrixDisplay

cm = metrics.confusion_matrix(y_test.values, y_pred[:,0])
disp = ConfusionMatrixDisplay(cm, display_labels=["Down", "Up"])
disp.plot()

Out[16]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at
0x77b37cfdald0>
```



3.2 Trading strategy based on model predictions

Next, let's evaluate the extent to which our model predictions can act as the foundation of a trading strategy. To that end, we will perform a **backtest** in the same spirit of Module 7 in Machine Learning:

```
In [17]: df_predictions = pd.DataFrame(
    {
        'Date': test_time, # No flatten needed for DatetimeIndex
        'Pred': y_pred.flatten() if y_pred.ndim > 1 else y_pred, #
        'Ret': Ret_vector.flatten() if Ret_vector.ndim > 1 else
    Ret_vector, # Flatten if 2D
    )
df_predictions.tail()
```

Out[17]:

	Date	Pred	Ret
1602	2025-04-30	0	0.61
1603	2025-05-01	0	0.39
1604	2025-05-02	0	-3.74
1605	2025-05-05	0	-3.15
1606	2025-05-06	0	-0.19

We will define the positions that our trading strategy will take as long (+1) if the prediction of the model is higher than 0.5, and short (−1) if less. *(Note, nonetheless, that in practice our prediction can only take values 0 or 1)*

As in previous cases, we will backtest 3 trading strategies:

- A long/short strategy that will take a long or short position when model prediction indicate so.
- A long-only strategy that will go to cash (return = 0) when model predicts a negative 120-day return.
- A Buy-and-hold strategy that will buy the stock at the beginning of the test period and hold it until the end of the period.

```
In [18]: df = df_predictions

df['Ret'] = df['Ret'] / 100

df["Positions"] = np.where(df["Pred"] > 0.5, 1, -1)
df["Strat_ret"] = df["Positions"].shift(1) * df["Ret"]
df["Positions_L"] = df["Positions"].shift(1)
df["Positions_L"][df["Positions_L"] == -1] = 0
df["Strat_ret_L"] = df["Positions_L"] * df["Ret"]
df["CumRet"] = df["Strat_ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)
df["CumRet_L"] = df["Strat_ret_L"].expanding().apply(lambda x: np.prod(1 + x) - 1)
df["bhRet"] = df["Ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)

Final_Return_L = np.prod(1 + df["Strat_ret_L"]) - 1
Final_Return = np.prod(1 + df["Strat_ret"]) - 1
Buy_Return = np.prod(1 + df["Ret"]) - 1

print("Strat Return Long Only =", Final_Return_L * 100, "%")
print("Strat Return =", Final_Return * 100, "%")
print("Buy and Hold Return =", Buy_Return * 100, "%")
```

/tmp/ipykernel_5841/2685361883.py:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df["Positions_L"][df["Positions_L"] == -1] = 0

Strat Return Long Only = 313.1173394983836 %

Strat Return = 194.30818715943187 %

Buy and Hold Return = 376.11163150377774 %

In [19]: `import matplotlib.pyplot as plt`

```
fig = plt.figure(figsize=(12, 6))
ax = plt.gca()
df.plot(x="Date", y="bhRet", label="Buy&Hold", ax=ax)
df.plot(x="Date", y="CumRet_L", label="Strat Only Long", ax=ax)
df.plot(x="Date", y="CumRet", label="Strat Long/Short", ax=ax)
plt.xlabel("date")
plt.ylabel("Cumulative Returns")
plt.grid()
plt.show()

df.describe()
```



Out[19]:

	Date	Pred	Ret	Posit ions	Strat _ret	Posit ions_ L	Strat _ret_ L	Cum Ret	Cum Ret_ L	bhRe t
count	1607	1607. 0000 00	1607. 0000 00	1607. 0000 00	1606. 0000 00	1606. 0000 00	1606. 0000 00	1606. 0000 00	1606. 0000 00	1607. 0000 00
mean	2022- 02-18 12:06 :43.2 3584 3328	0.842 564	0.001 174	0.685 128	0.000 877	0.843 088	0.001 037	0.976 935	1.700 314	2.328 598
min	2018- 12-07 00:00 :00	0.000 000	-0.12 8600	-1.00 0000	-0.15 3300	0.000 000	-0.12 8600	-0.15 6117	-0.15 6117	-0.18 6244
25%	2020- 07-15 12:00 :00	1.000 000	-0.00 8150	1.000 000	-0.00 8300	1.000 000	-0.00 5900	0.531 523	0.791 044	1.225 672
50%	2022- 02-16 00:00 :00	1.000 000	0.001 500	1.000 000	0.001 150	1.000 000	0.000 000	0.897 106	1.677 941	2.488 385
75%	2023- 09-26 12:00 :00	1.000 000	0.011 900	1.000 000	0.011 600	1.000 000	0.009 000	1.613 472	2.541 385	3.233 868
max	2025- 05-06 00:00 :00	1.000 000	0.153 300	1.000 000	0.119 800	1.000 000	0.119 800	2.506 969	3.963 271	5.205 463
std	NaN	0.364 325	0.020 169	0.728 650	0.020 171	0.363 831	0.017 512	0.677 663	1.050 868	1.368 053

4. Conclusion

In this lesson, we have worked with TensorFlow Keras on a MLP model for classification purposes. Specifically, we designed a MLP model that aims to predict the forward 120 day return of Apple stock. In doing so, we have introduced some novel features of Deep Learning models, such as dropout and class weights, which will also come in handy later on for improving the performance of our models. Still, there is much to be done in terms of improving model performance by, for example, optimally selecting some of the hyperparameters used in the model such as the learning rate. That is what we will do in the remainder of the module introducing hyperparameter tuning. For that, see you in the next lesson!

Required Readings

Required readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Tutorial "Introduction to Keras Tuner" https://www.tensorflow.org/tutorials/keras/keras_tuner?hl=en.

- **Required pages:** Entire tutorial
- **Estimated reading time:** 1 hour

2. Hutter, Frank, et al. *Automated Machine Learning*. <https://link.springer.com/book/10.1007/978-3-030-05318-5>

- **Required pages:** Chapter 1
- **Estimated reading time:** 30 minutes

3. Yu, Tong, et al. "Hyper-Parameter Optimization: A Review of Algorithms and Applications." [\[https://arxiv.org/abs/2003.05689\]](https://arxiv.org/abs/2003.05689).

- **Required pages:** Section 1 to 3
- **Estimated reading time:** 50 minutes

4. Yang, Li, et al. "On Hyperparameter Optimization of Machine Learning Algorithms: Theory and Practice." [\[https://arxiv.org/abs/2007.15745\]](https://arxiv.org/abs/2007.15745)

- **Required pages:** Section 4
- **Estimated reading time:** 30 minutes

Module 1: Lesson 3

Hyperparameter Optimization (HPO)

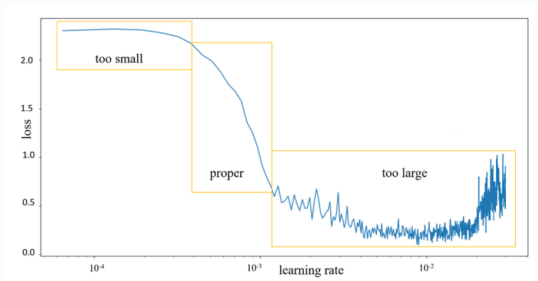


Outline

- ▶ Hyperparameter Optimization (HPO): why, what, and how?
- ▶ Which hyperparameters?
- ▶ Optimization algorithms

Hyperparameter Optimization (HPO)

- ▶ So far, we have used neural networks in a kind of **brute force** approach: simply by using trial and error in our models, we would see which combination of features (i.e., number of hidden layers, learning rate, optimizer, etc.) perform best.
- ▶ This is however quite inefficient because it requires spending a lot of time deciphering the best choices for a model.
- ▶ **Solution:** Hyperparameter Optimization (HPO)



Hyperparameter Optimization (HPO)

► **Solution:** Hyperparameter Optimization (HPO)

- Hyperparameters refer to parameters that cannot be updated during the training of the model.
- These hyperparameters can be **structure-related** (e.g., # of hidden layers), or **training-related** (e.g., learning rate or minibatch size)
- Final step of model design and first step of training a neural network

► **Purpose of HPO:**

1. Reduce costly menial work by the researcher
2. Improve accuracy and efficiency of training
3. Make the choice of hyper-parameters more convincing and reproducible

► **How?**

- We'll be using a series of optimization techniques
- These can be classified as **search algorithms** (e.g., grid search) and **trial algorithms** (e.g., curve fitting).

Which hyperparameters?

Given that HPO takes considerable computational resources, it is important to understand which hyperparameters take preferential treatment in the tuning. Here, we review major hyperparameters based on previous researchers' experience.

► Training-related hyperparameters:

- Learning rate (constant, linear/exponential decay)
- Optimizer (Mini-batch SGD, RMSprop, Adam)
- ...

► Model-design hyperparameters:

- Number of hidden layers
- Width (number of neurons) of hidden layers
- Regularization in cost function (ℓ_1 vs. ℓ_2 norms)
- Dropout (which rate?)
- Activation function (ReLU, Sigmoid, Tanh?)
- ...

Optimization algorithms

- Mathematically, HPO consists simply of finding a set of hyperparameters that achieve the minimum loss (or maximum accuracy) of a network/model.
- Computationally, HPO represents a complex computational problem to be solved by state-of-the-art algorithms of mainly two types: search and trial.

► Search algorithms

- Grid search: straightforward, but computationally costly (remember the curse of dimensionality!)
- Random search: similar to grid with randomized search (less time-consuming)
- Bayesian Optimization: selects next hyperparameter based on previous experience

► (Trial) Early-stopping algorithms

- Curve fitting: LPA algorithm (Learn, Predict, Assess)
- Successive Halving (SHA) and Hyperband (HB): random search sampling method + bandit-based early-stopping policy
- Extensions: Asynchronous Successive Halving (ASHA) and Bayesian Optimization Hyperband (BOHB)

Summary of Lesson 3

In Lesson 3, we have looked at:

- ▶ What is HPO, why we need it, and the basics of how to perform it
- ▶ Main hyper-parameters we can tune
- ▶ Basic review of algorithms for HPO

⇒ **References for this lesson:**

Yu, Tong, and Hong Zhu. "Hyperparameter Optimization: A Review of Algorithms and Applications." arXiv, 2020, <https://arxiv.org/abs/2003.05689>

. **TO-DO NEXT:** There is no associated Jupyter Notebook for this lesson, but it is extremely important that you go over the required readings for this lesson thoroughly.

In the next lesson, we will come back one more time to our stock timing example to see how a regression model based on a somewhat simple neural network can benefit from hyperparameter tuning using the Keras Tuner.

Required Readings

Required readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Zhang, Ashton, et al. *Dive into Deep Learning*. <https://d2l.ai/d2l-en.pdf>.

- **Required pages:** Section 3.7 and Sections 11.4, 11.5, 11.6, 11.7, 11.8, 11.9, 11.10, and 11.11
- **Estimated reading time:** 1 hour 15 minutes

Module 1: Lesson 4

Hyperparameter Optimization on Keras Tuner



Outline

- ▶ Regression problem: Testing the Keras tuner
- ▶ Keras Tuner
- ▶ Timing AAPL stock using tuned model hyperparameters

Testing the Keras Tuner

In this practical lesson, we'll revisit the problem of timing AAPL stock (or any other stock/financial time series for that matter) that we faced in lesson 2 of this module. This time, however, we'll make it a regression (rather than classification) problem:

- ▶ To do this, we will use an MLP framework in a regression problem to predict the future 25-day of AAPL stock
- ▶ Different from previous examples, here, we will incorporate hyper-parameter tuning via the Keras Tuner.
- ▶ This whole exercise yields two main takeaways:
 - (i) Hyperparameter tuning is **computationally costly** (even in our simple case, it takes up to 5 minutes of computations).
 - (ii) Hyperparameter tuning can have a **substantial effect** on the predictive performance of our models (even simple tuning), especially in these first initial stages for us.
- ▶ For the complete documentation on the Keras tuner, please visit: https://keras.io/api/keras_tuner/

Keras Tuner

The HPO task is much facilitated by the Keras tuner: a library for hyperparameter tuning in TensorFlow.

- ▶ To properly use it, we first need to define our **hypermodel**: which parameters should we tune? what choices are available?
- ▶ In this case, we will tune: (i) # hidden layers, (ii) # units in hidden layers, and (iii) dropout rate:

```
class MLP_model(HyperModel):
    def build(self, hp):

        # We define a constant activation function of ReLU form. We will not be tuning the activation functions
        act_fun = 'relu'

        # We do ask the Keras Tuner to choose whether is best to have a dropout rate after each hidden layer of 0.2 or 0.3
        n_dropout = hp.Choice('n_dropout', values=[0.20, 0.30])

        model = tf.keras.models.Sequential()

        # Now, we will use a loop to let the tuner choose the number of layers that is best for the model between 1 and 5
        for i in range(1, hp.Int("num_layers", 1, 5)):
            # Within this loop, we will also ask the tuner to decide the optimal number of units that each of the selected layer should have.
            model.add(tf.keras.layers.Dense(units= hp.Int("units_dense_" + str(i), min_value=1, max_value=25, step=5), activation=act_fun))
            model.add( tf.keras.layers.Dropout(n_dropout))

        model.add(tf.keras.layers.Dense(units = 1))

        # As was the case with the activation function, there is no tuning on the learning rate, nor the optimizer, although we could
        # easily incorporate it.
        hp_lr = 1e-5
        adam = tf.keras.optimizers.Adam(learning_rate = hp_lr)
        model.compile(optimizer = adam, loss='mean_absolute_error')

        return model
```

Keras Tuner

Once we have built our hypermodel, it's time to put it to work:

- Our choice of optimizer (i.e., tuner) will be the **hyperband**:

Li, Lisha, et al. "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization." *The Journal of Machine Learning Research*, vol. 18, no. 1, 2017, pp. 6765–6816, <https://www.jmlr.org/papers/volume18/16-558/16-558.pdf>

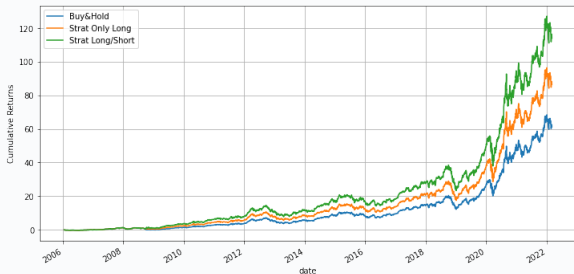
- Hyperband requires:
 - Hypermodel: just defined!
 - Objective: how to optimize? → Minimum loss in validation
 - Other parameters: setting random seed, storage of results, etc.

```
hypermodel=MLP_model()
tuner = kt.Hyperband(hypermodel,
                    objective = kt.Objective("val_loss", direction="min"),
                    overwrite = True,
                    max_epochs = 30,
                    seed = 1234,
                    directory=os.path.normpath('C:/'))

# Let's run the tuner! (Warning: this could take time)
tuner.search(X_train, y_train, validation_split=val_split)
best_hps = tuner.get_best_hyperparameters(num_trials = 1)[0]
```

Trading strategy based on tuned model predictions

Once we have tuned our model hyperparameters, we can check how a trading strategy based on model predictions in the test sample would have performed via **backtesting**:



One more time, please do remember the caveats associated with backtesting:

- It only validates the performance of the strategy for the test sample—no external validity from this!

Summary of Lesson 4

In Lesson 4, we have looked at:

- ▶ Keras tuner for hyperparameter optimization
- ▶ Performance of Keras tuner in a regression problem

⇒ **References for this lesson:**

Yu, Tong, and Hong Zhu. "Hyperparameter Optimization: A Review of Algorithms and Applications." *arXiv*, 2020, <https://arxiv.org/abs/2003.05689>

Introduction to Keras tuner example in TensorFlow:

https://www.tensorflow.org/tutorials/keras/keras_tuner?hl=en

TO DO NEXT: Go to the associated **Jupyter Notebook** to cover the full content of this lesson.

In the next module, we'll continue to dig deeper into the Deep Learning world by moving forward to more advanced and complex models and techniques.

DEEP LEARNING FOR FINANCE

MODULE 1 | LESSON 4

Multilayer Perceptron: Revisiting market Timing of APPL stock with Hyper-parameter Tuning

Reading Time	60 minutes
Prior Knowledge	Deep Learning, MLP, Timing Strategy, Hyper-parameter
Keywords	MLP for Regression, Hyper-parameter Tuning, Keras Tuner , Hyperband

In this lesson we revisit the timing our investments in AAPL stock under a regression problem using MLPs with Hyper-parameter tuning. This way, we will be able to observe first-hand the power of Hyper-parameter optimization (HPO). In order to complicate the predictability problem a bit over the classification example we went over in Lesson 2, and also to conveniently use the simplest form of the Keras Tuner, our predictive model MLP will be a regression one. As usual, we will first focus on the Deep Learning side of the task by building a predictive regression model with HPO, and later on check the performance of such a trading strategy in the test sample.

1. Data and sources

As we did in the example of Lesson 2, we will keep using the **yfinance** library to retrieve daily Apple stock price data from Yahoo! Finance:

```
In [1]: import numpy as np
import pandas as pd

# Read the CSV file
file_path = 'AAPL.csv'
df = pd.read_csv(file_path, delimiter=';', decimal=',', parse_dates=
['Date'], date_format='%m/%d/%y')
df = df.sort_values('Date')
df = df.set_index('Date')

# Rename '% Return' column to 'Ret'
df = df.rename(columns={'% Return': 'Ret'})

# Define the name variable
name = 'Ret'

# Set 'Date' as index and keep only 'Ret' column
df = df.dropna()

df.head()
```

```
Out[1]:
```

	Price	CVol	Change	% Change	Ret	Total Return (Gross)	Cumulative Return %	Open	High	Low	Cumulative Change %
Date											
1991-12-02	0.46	118899320.0	0.01	1.97	1.97	0.48	276,80	0.45	0.46	0.45	260,13
1991-12-03	0.45	102860910.0	-0.01	-2.42	-2.42	0.47	267,70	0.46	0.46	0.45	251,44
1991-12-05	0.45	99198490.0	0.00	-0.99	-0.99	0.47	264,06	0.45	0.46	0.44	247,96
1991-12-06	0.44	196985780.0	-0.01	-2.50	-2.50	0.46	254,96	0.44	0.44	0.43	239,26
1991-12-09	0.44	97832080.0	0.00	0.77	0.77	0.46	257,69	0.44	0.45	0.44	241,87

2. Timing Apple stock with Multilayer Perceptron (MLPs)

Now, let's start building our model in a similar fashion to what we have done in previous examples, starting with inputs and output variables:

2.1 Inputs and outputs

For, among others, comparability reasons, we will consider the very same inputs for the regression model as we did for the classification example of Lesson 2. Specifically, those are AAPL stock returns from the past 25, 60, 90, 120 and 240 days:

```
In [2]: df = df[['Ret']]

df["Ret25_i"] = df[name].rolling(25).apply(lambda x: 100 * (np.prod(1 + x / 100) - 1))
df["Ret60_i"] = df[name].rolling(60).apply(lambda x: 100 * (np.prod(1 + x / 100) - 1))
df["Ret90_i"] = df[name].rolling(90).apply(lambda x: 100 * (np.prod(1 + x / 100) - 1))
df["Ret120_i"] = df[name].rolling(120).apply(lambda x: 100 * (np.prod(1 + x / 100) - 1))
df["Ret240_i"] = df[name].rolling(240).apply(lambda x: 100 * (np.prod(1 + x / 100) - 1))

df = df.dropna()
df.tail(10)
```

```
Out[2]:
```

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i
2025-04-23	2.43	-3.795072	-10.881083	-17.324816	-11.375229	13.166308
2025-04-24	1.84	-3.186662	-12.437332	-15.363482	-10.514112	14.812281
2025-04-25	0.44	-2.242569	-12.454764	-15.498093	-10.228100	15.098768
2025-04-28	0.41	-3.719238	-11.440489	-15.210988	-8.459465	14.426408
2025-04-29	0.51	-4.309509	-10.388438	-15.764123	-6.287033	15.809066
2025-04-30	0.61	-5.026928	-6.678198	-16.064459	-4.444495	14.500296
2025-05-01	0.39	-3.703195	-8.241178	-13.894452	-3.686575	14.238568
2025-05-02	-3.74	-8.267882	-11.549127	-17.690963	-7.887429	8.640630
2025-05-05	-3.15	-8.729653	-14.608582	-21.754709	-10.493604	5.155357
2025-05-06	-0.19	-10.636714	-12.675026	-22.144726	-12.535408	4.934575

- **Defining the output: Regression**

Then, we need to define our output variable. As we have already mentioned, we will face a regression problem here so our output is actually a continuous

variable containing the returns of AAPL stock for some future time span.

In this case, our model will aim to predict the return of AAPL stock over the next **25 days**. Please note that this choice is completely subjective (the model may even work better for some other time span!). Once you have gone through the entire notebook and understood the basics of the process, please go ahead and play around with all these features! All rights reserved WQU WorldQuant University QQQQ

```
In [3]: df["Ret25"] = df["Ret25_i"].shift(-25)
df = df.dropna()
df.tail(10)
```

```
Out[3]:
```

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i	Ret25
Date							
2025-03-18	-0.61	-6.554621	-15.996749	-4.367412	-6.574936	25.694524	-3.795072
2025-03-19	1.20	-7.450848	-13.129686	-2.831145	-4.729781	28.099556	-3.186662
2025-03-20	-0.53	-9.595756	-14.190763	-3.970333	-5.612264	26.811931	-2.242569
2025-03-21	1.95	-9.613488	-14.131805	-1.773607	-3.346428	29.921378	-3.719238
2025-03-24	1.13	-9.738442	-13.429862	-2.744908	-2.750216	30.800886	-4.309509
2025-03-25	1.37	-8.456087	-13.241573	-1.294066	-1.536051	33.487222	-5.026928
2025-03-26	-0.99	-9.507160	-14.374483	-1.084266	-4.693366	31.220908	-3.703195
2025-03-27	1.05	-8.912227	-12.318013	-0.443875	-0.806104	34.087094	-8.267882
2025-03-28	-2.66	-11.237523	-13.499902	-4.411194	-3.685448	25.103400	-8.729653
2025-03-31	1.94	-10.082015	-11.191258	-1.163172	-1.333480	26.442997	-10.636714

```
In [4]: df.describe()
```

Out[4]:

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i	Ret25
count	8010.00 0000	8010.00 0000	8010.00 0000	8010.00 0000	8010.00 0000	8010.00 0000	8010.00 0000
mean	0.11521 1	2.91992 3	7.32560 7	11.2290 95	15.1969 86	32.6242 28	2.86905 3
std	2.69197 5	13.3897 84	21.6023 10	27.5674 21	33.1220 47	56.1017 87	13.4133 49
min	-51.8700 00	-69.5981 32	-77.0749 30	-72.4838 02	-74.7292 50	-75.0073 29	-69.5981 32
25%	-1.16000 0	-4.79123 1	-4.84300 4	-4.88483 4	-3.92448 1	-6.06395 0	-4.87959 4
50%	0.10000 0	2.89412 2	7.11433 9	10.7138 40	14.7133 42	29.2385 60	2.81247 4
75%	1.37000 0	10.5871 34	19.2632 31	26.1273 22	30.7363 67	59.1769 53	10.5726 72
max	33.2300 00	121.286 374	116.442 841	135.727 260	167.513 049	300.683 525	121.286 374

2.2 Train-Test samples and Scaling

The next important step, as you know, is in defining our train and test samples, together with the scaling of variables.

Now, as you may remember, we did not perform any scaling in the example from Lesson 2, and that is because all of the input and output variables are essentially of the same nature (i.e., stock returns), but note that it is common (and good) practice to perform the scaling. You can incorporate the scaling here by relying on what we did in the last module of ML. Indeed, you can actually try check whether different scaling methods matter for the predictions.

As for the definition of test and train samples, we will devote the usual first 60% of examples in that data (in chronological order) to training and the remaining 40% to testing:

```
In [5]: # Train-test split
ts = int(0.4 * len(df)) # Number of observations in test sample (20%)
split_time = len(df) - ts # Start of test sample
test_time = df.index[split_time:] # Extract Date index for test period
Ret_vector = df['Ret'][split_time:].values # Extract Ret values for test
period
df.tail()
```

Out[5]:

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i	Ret25
Date							
2025-03-25	1.37	-8.456087	-13.241573	-1.294066	-1.536051	33.487222	-5.026928
2025-03-26	-0.99	-9.507160	-14.374483	-1.084266	-4.693366	31.220908	-3.703195
2025-03-27	1.05	-8.912227	-12.318013	-0.443875	-0.806104	34.087094	-8.267882
2025-03-28	-2.66	-11.237523	-13.499902	-4.411194	-3.685448	25.103400	-8.729653
2025-03-31	1.94	-10.082015	-11.191258	-1.163172	-1.333480	26.442997	-10.636714

So let's formally split our data in train and test samples for input and output variables:

```
In [6]: from sklearn.model_selection import train_test_split

Xdf, ydf = df.iloc[:, 1:-1], df.iloc[:, -1]
X = Xdf.astype("float32")
y = ydf.astype("float32")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=ts, shuffle=False
) # It is important to keep "shuffle=False"
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(4806, 4) (3204, 4) (4806,) (3204,)
```

2.3 Model and Training

Finally, it is time to set up our model. This is where the new things in the lesson come, which mainly sum up to one: **Keras Tuner**.

- **Keras Tuner**

Keras Tuner is a library that helps us in Hyper-parameter tuning for our TensorFlow programs. The functioning of the library is fairly easy and intuitive, with some specific syntax that you should be aware of. Despite that we will be using the Keras Tuner in this and future notebooks, we strongly recommend that you take a look at some extra resources:

- This simple tutorial from the TensorFlow website:
https://www.tensorflow.org/tutorials/keras/keras_tuner?hl=en
- There are multiple online examples and tutorials on how to use the Keras Tuner, here you have one that you can consult for clarification:
<https://www.youtube.com/watch?v=6Nf1x7qThR8>
- Keras Tuner **library documentation**: https://keras.io/api/keras_tuner/

Now the Keras Tuners works in a rather simple way, but it requires us to perform tasks in an organized way. Let's see how.

First we will need to import the library, of course:

```
In [7]: !pip install keras_tuner
```

```
Requirement already satisfied: keras_tuner in
/usr/local/lib/python3.11/site-packages (1.4.7)
Requirement already satisfied: keras in /usr/local/lib/python3.11/site-
packages (from keras_tuner) (3.9.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/site-
packages (from keras_tuner) (23.1)
Requirement already satisfied: requests in /usr/local/lib/python3.11/site-
packages (from keras_tuner) (2.32.3)
Requirement already satisfied: kt-legacy in /usr/local/lib/python3.11/site-
packages (from keras_tuner) (1.0.5)
Requirement already satisfied: absl-py in /usr/local/lib/python3.11/site-
packages (from keras->keras_tuner) (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/site-
packages (from keras->keras_tuner) (1.26.2)
Requirement already satisfied: rich in /usr/local/lib/python3.11/site-
packages (from keras->keras_tuner) (14.0.0)
Requirement already satisfied: namex in /usr/local/lib/python3.11/site-
packages (from keras->keras_tuner) (0.0.9)
Requirement already satisfied: h5py in /usr/local/lib/python3.11/site-
packages (from keras->keras_tuner) (3.13.0)
Requirement already satisfied: optree in /usr/local/lib/python3.11/site-
packages (from keras->keras_tuner) (0.15.0)
Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.11/site-
packages (from keras->keras_tuner) (0.5.1)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.11/site-packages (from requests->keras_tuner) (3.1.0)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.11/site-packages (from requests->keras_tuner) (3.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.11/site-packages (from requests->keras_tuner)
(1.26.15)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.11/site-packages (from requests->keras_tuner)
(2022.12.7)
Requirement already satisfied: typing-extensions>=4.5.0 in
/usr/local/lib/python3.11/site-packages (from optree->keras->keras_tuner)
(4.13.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.11/site-packages (from rich->keras->keras_tuner)
(3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.11/site-packages (from rich->keras->keras_tuner)
(2.15.1)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/site-
packages (from markdown-it-py>=2.2.0->rich->keras->keras_tuner) (0.1.2)
WARNING: Running pip as the 'root' user can result in broken permissions and
conflicting behaviour with the system package manager. It is recommended to
use a virtual environment instead: https://pip.pypa.io/warnings/venv
```

```
[notice] A new release of pip available: 22.3.1 -> 25.1
```

```
[notice] To update, run: pip install --upgrade pip
```

```
In [8]: import keras_tuner as kt
import tensorflow as tf
from keras_tuner import HyperModel
```

```
2025-05-07 07:46:50.146413: I tensorflow/core/util/port.cc:153] oneDNN
custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-05-07 07:46:50.195517: I
tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is
optimized to use available CPU instructions in performance-critical
operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16
AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
2025-05-07 07:46:51.312736: I tensorflow/core/util/port.cc:153] oneDNN
custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

Then, in addition to setting the random seed for replicability and the validation split for the use of early-stopping later on, we need to define a function **build()** within the **MLP_model** class.

This build function contains the different 'choices' we are going to ask the Keras Tuner to decide for us. In this first and simple example we are simply tuning 3 hyper-parameters:

- **Dropout:** we are letting the tuner choose between a value for dropout of 0.2 or 0.3.
- **Number of layers in the model:** in this case, we will let the tuner choose between 1 and 5 hidden layers in the model. We do this using the **hp.Int()** method.
- **Number of units in each hidden layer:** the tuner will choose between 1 (min_value) and 25 (max_value) units in each layer, with a step size of 5.

```

In [9]: tf.keras.backend.clear_session()
tf.random.set_seed(1234)
val_split = 0.2

class MLP_model(HyperModel):
    def build(self, hp):
        # We define a constant activation function of ReLU form. We will
        # not be tuning the activation functions
        act_fun = "relu"

        # We do ask the Keras Tuner to choose whether is best to have a
        # dropout rate after each hidden layer of 0.2 or 0.3
        n_dropout = hp.Choice("n_dropout", values=[0.20, 0.30])

        model = tf.keras.models.Sequential()

        # Now, we will use a loop to let the tuner choose the number of
        # layers that is best for the model between 1 and 5
        for i in range(1, hp.Int("num_layers", 1, 5)):
            # Within this loop, we will also ask the tuner to decide the
            # optimal number of units that each of the selected layer should have.
            model.add(
                tf.keras.layers.Dense(
                    units=hp.Int(
                        "units_dense_" + str(i), min_value=1,
max_value=25, step=5
                    ),
                    activation=act_fun,
                )
            )
            model.add(tf.keras.layers.Dropout(n_dropout))

        model.add(tf.keras.layers.Dense(units=1))

        # As was the case with the activation function, there is no tuning
        # on the learning rate, nor the optimizer, although we could easily
        # incorporate it.
        hp_lr = 1e-5
        adam = tf.keras.optimizers.Adam(learning_rate=hp_lr)
        model.compile(optimizer=adam, loss="mean_absolute_error")

        return model

```

Now, once we have defined our HyperModel (i.e., **MLP_model()** class), we can perform the tuning of the hyper-parameters. This is, essentially, another optimization algorithm (**Tuner**), for which we would also need to make some decisions:

- **Hypermodel:** we have already defined our hypermodel: **MLP_model()**
- **Objective:** what is the objective function to optimize? In this case we will minimize the loss function in the validation set.
- **Other parameters:** these include whether we overwrite results, fixing the random seed, or storage of results (convenient for large complex models).

But what about the **Tuner**? In this case, we will select the Hyperband tuner embedded in Keras, for which you can consult the documentation here: https://keras.io/api/keras_tuner/tuners/hyperband/#hyperband-class

Arguably, you could easily have decided to use another kind of optimizer from the ones available in the Keras Tuner (https://keras.io/api/keras_tuner/tuners/). Feel free to come back here once you have completed the notebook and see what would happen under another tuner!

```
In [10]: import os

from keras import backend as K

# First, we clear the session just to make sure our seeds are correctly
working and replicability is achieved
K.clear_session()

# Then, we call the model and perform the tuning:
hypermodel = MLP_model()
tuner = kt.Hyperband(
    hypermodel,
    objective=kt.Objective("val_loss", direction="min"),
    overwrite=True,
    max_epochs=10,
    seed=1234,
    directory=os.path.normpath("C:/"),
)

# Let's run the tuner! (Warning: this could take time)
tuner.search(X_train, y_train, validation_split=val_split)
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]
```

Trial 30 Complete [00h 00m 05s]

val_loss: 8.497085571289062

Best val_loss So Far: 7.930953025817871

Total elapsed time: 00h 01m 14s

```
In [11]: best_hps.values
```

```
Out[11]: {'n_dropout': 0.2,
          'num_layers': 3,
          'units_dense_1': 11,
          'units_dense_2': 21,
          'units_dense_3': 11,
          'units_dense_4': 21,
          'tuner/epochs': 10,
          'tuner/initial_epoch': 4,
          'tuner/bracket': 1,
          'tuner/round': 1,
          'tuner/trial_id': '0021'}
```

Now, we can use these tuned hyper-parameter values to train our model, together with a usual early-stopping criteria:

```
In [12]: model = tuner.hypermodel.build(best_hps)
es = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss", mode="min", verbose=1, patience=20,
    restore_best_weights=True
)

# fit the model
model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=200,
    batch_size=32,
    verbose=2,
    callbacks=[es],
)
```

Epoch 1/200
121/121 - 1s - 8ms/step - loss: 60.7589 - val_loss: 55.2892
Epoch 2/200
121/121 - 1s - 4ms/step - loss: 60.5621 - val_loss: 54.1300
Epoch 3/200
121/121 - 0s - 2ms/step - loss: 57.7094 - val_loss: 53.0120
Epoch 4/200
121/121 - 0s - 3ms/step - loss: 57.6723 - val_loss: 51.8763
Epoch 5/200
121/121 - 0s - 2ms/step - loss: 56.7486 - val_loss: 50.7455
Epoch 6/200
121/121 - 0s - 3ms/step - loss: 54.7629 - val_loss: 49.6327
Epoch 7/200
121/121 - 0s - 2ms/step - loss: 54.0818 - val_loss: 48.5352
Epoch 8/200
121/121 - 0s - 3ms/step - loss: 52.7764 - val_loss: 47.4433
Epoch 9/200
121/121 - 0s - 3ms/step - loss: 52.4463 - val_loss: 46.3664
Epoch 10/200
121/121 - 0s - 2ms/step - loss: 51.0419 - val_loss: 45.2981
Epoch 11/200
121/121 - 0s - 2ms/step - loss: 49.1633 - val_loss: 44.2710
Epoch 12/200
121/121 - 0s - 2ms/step - loss: 48.2504 - val_loss: 43.2420
Epoch 13/200
121/121 - 0s - 3ms/step - loss: 47.6171 - val_loss: 42.2085
Epoch 14/200
121/121 - 0s - 2ms/step - loss: 46.6572 - val_loss: 41.1772
Epoch 15/200
121/121 - 0s - 3ms/step - loss: 46.3400 - val_loss: 40.1405
Epoch 16/200
121/121 - 0s - 2ms/step - loss: 44.4603 - val_loss: 39.1390
Epoch 17/200
121/121 - 0s - 2ms/step - loss: 43.9083 - val_loss: 38.1496
Epoch 18/200
121/121 - 0s - 2ms/step - loss: 42.4401 - val_loss: 37.1640
Epoch 19/200
121/121 - 0s - 3ms/step - loss: 42.3213 - val_loss: 36.1823
Epoch 20/200
121/121 - 0s - 2ms/step - loss: 41.1723 - val_loss: 35.2069
Epoch 21/200
121/121 - 0s - 3ms/step - loss: 40.4357 - val_loss: 34.2402
Epoch 22/200
121/121 - 0s - 2ms/step - loss: 39.6546 - val_loss: 33.2788
Epoch 23/200
121/121 - 0s - 3ms/step - loss: 37.9864 - val_loss: 32.3380
Epoch 24/200
121/121 - 0s - 2ms/step - loss: 36.7197 - val_loss: 31.4274
Epoch 25/200
121/121 - 0s - 3ms/step - loss: 36.6431 - val_loss: 30.4911
Epoch 26/200
121/121 - 0s - 2ms/step - loss: 35.4619 - val_loss: 29.5744
Epoch 27/200
121/121 - 0s - 2ms/step - loss: 35.1836 - val_loss: 28.6601
Epoch 28/200
121/121 - 0s - 3ms/step - loss: 33.9963 - val_loss: 27.7613
Epoch 29/200
121/121 - 0s - 2ms/step - loss: 33.8394 - val_loss: 26.8533
Epoch 30/200
121/121 - 0s - 2ms/step - loss: 32.4806 - val_loss: 25.9754

Epoch 31/200
121/121 - 0s - 2ms/step - loss: 32.4247 - val_loss: 25.0806
Epoch 32/200
121/121 - 0s - 3ms/step - loss: 30.8789 - val_loss: 24.2257
Epoch 33/200
121/121 - 0s - 2ms/step - loss: 30.3947 - val_loss: 23.3857
Epoch 34/200
121/121 - 0s - 2ms/step - loss: 29.3155 - val_loss: 22.5609
Epoch 35/200
121/121 - 0s - 2ms/step - loss: 29.9390 - val_loss: 21.7183
Epoch 36/200
121/121 - 0s - 2ms/step - loss: 28.6318 - val_loss: 20.9313
Epoch 37/200
121/121 - 0s - 3ms/step - loss: 28.5308 - val_loss: 20.1763
Epoch 38/200
121/121 - 0s - 2ms/step - loss: 27.4630 - val_loss: 19.4093
Epoch 39/200
121/121 - 0s - 2ms/step - loss: 27.7033 - val_loss: 18.6465
Epoch 40/200
121/121 - 0s - 3ms/step - loss: 26.6301 - val_loss: 17.8917
Epoch 41/200
121/121 - 0s - 2ms/step - loss: 26.6684 - val_loss: 17.1859
Epoch 42/200
121/121 - 0s - 2ms/step - loss: 25.9235 - val_loss: 16.4922
Epoch 43/200
121/121 - 0s - 3ms/step - loss: 25.6605 - val_loss: 15.8231
Epoch 44/200
121/121 - 0s - 2ms/step - loss: 25.1218 - val_loss: 15.2005
Epoch 45/200
121/121 - 0s - 2ms/step - loss: 24.3488 - val_loss: 14.6080
Epoch 46/200
121/121 - 0s - 2ms/step - loss: 24.7702 - val_loss: 14.0242
Epoch 47/200
121/121 - 0s - 3ms/step - loss: 24.3320 - val_loss: 13.4639
Epoch 48/200
121/121 - 0s - 2ms/step - loss: 24.0687 - val_loss: 12.9288
Epoch 49/200
121/121 - 0s - 3ms/step - loss: 23.4684 - val_loss: 12.5103
Epoch 50/200
121/121 - 0s - 2ms/step - loss: 23.1510 - val_loss: 12.1158
Epoch 51/200
121/121 - 0s - 3ms/step - loss: 22.9878 - val_loss: 11.7210
Epoch 52/200
121/121 - 0s - 2ms/step - loss: 22.4297 - val_loss: 11.4079
Epoch 53/200
121/121 - 0s - 3ms/step - loss: 22.7052 - val_loss: 11.1035
Epoch 54/200
121/121 - 0s - 2ms/step - loss: 22.3456 - val_loss: 10.7939
Epoch 55/200
121/121 - 0s - 2ms/step - loss: 22.7060 - val_loss: 10.5122
Epoch 56/200
121/121 - 0s - 2ms/step - loss: 21.8393 - val_loss: 10.2943
Epoch 57/200
121/121 - 0s - 3ms/step - loss: 21.8695 - val_loss: 10.1145
Epoch 58/200
121/121 - 0s - 2ms/step - loss: 21.8387 - val_loss: 9.9440
Epoch 59/200
121/121 - 0s - 2ms/step - loss: 21.2471 - val_loss: 9.7854
Epoch 60/200
121/121 - 0s - 3ms/step - loss: 21.5242 - val_loss: 9.6590

Epoch 61/200
121/121 - 0s - 2ms/step - loss: 21.3108 - val_loss: 9.5461
Epoch 62/200
121/121 - 0s - 3ms/step - loss: 20.8671 - val_loss: 9.4506
Epoch 63/200
121/121 - 0s - 2ms/step - loss: 20.9935 - val_loss: 9.3523
Epoch 64/200
121/121 - 0s - 2ms/step - loss: 20.7324 - val_loss: 9.2702
Epoch 65/200
121/121 - 0s - 3ms/step - loss: 21.1063 - val_loss: 9.2183
Epoch 66/200
121/121 - 0s - 2ms/step - loss: 20.9318 - val_loss: 9.1708
Epoch 67/200
121/121 - 0s - 2ms/step - loss: 20.6103 - val_loss: 9.1311
Epoch 68/200
121/121 - 0s - 3ms/step - loss: 20.2585 - val_loss: 9.1028
Epoch 69/200
121/121 - 0s - 2ms/step - loss: 20.4676 - val_loss: 9.0806
Epoch 70/200
121/121 - 0s - 2ms/step - loss: 20.4251 - val_loss: 9.0507
Epoch 71/200
121/121 - 0s - 3ms/step - loss: 20.2054 - val_loss: 9.0271
Epoch 72/200
121/121 - 0s - 2ms/step - loss: 20.3319 - val_loss: 9.0065
Epoch 73/200
121/121 - 0s - 2ms/step - loss: 20.0951 - val_loss: 8.9848
Epoch 74/200
121/121 - 0s - 2ms/step - loss: 19.8333 - val_loss: 8.9672
Epoch 75/200
121/121 - 0s - 3ms/step - loss: 19.8877 - val_loss: 8.9504
Epoch 76/200
121/121 - 0s - 2ms/step - loss: 20.1308 - val_loss: 8.9287
Epoch 77/200
121/121 - 0s - 2ms/step - loss: 20.1989 - val_loss: 8.9140
Epoch 78/200
121/121 - 0s - 2ms/step - loss: 19.7295 - val_loss: 8.9034
Epoch 79/200
121/121 - 0s - 2ms/step - loss: 19.5249 - val_loss: 8.8879
Epoch 80/200
121/121 - 0s - 2ms/step - loss: 19.8580 - val_loss: 8.8795
Epoch 81/200
121/121 - 0s - 2ms/step - loss: 19.3913 - val_loss: 8.8655
Epoch 82/200
121/121 - 0s - 2ms/step - loss: 19.5088 - val_loss: 8.8486
Epoch 83/200
121/121 - 0s - 3ms/step - loss: 19.7102 - val_loss: 8.8317
Epoch 84/200
121/121 - 0s - 2ms/step - loss: 19.0490 - val_loss: 8.8178
Epoch 85/200
121/121 - 0s - 2ms/step - loss: 19.2517 - val_loss: 8.8017
Epoch 86/200
121/121 - 0s - 2ms/step - loss: 19.4366 - val_loss: 8.7883
Epoch 87/200
121/121 - 0s - 2ms/step - loss: 19.2342 - val_loss: 8.7759
Epoch 88/200
121/121 - 0s - 2ms/step - loss: 19.0456 - val_loss: 8.7625
Epoch 89/200
121/121 - 0s - 3ms/step - loss: 19.2636 - val_loss: 8.7499
Epoch 90/200
121/121 - 0s - 3ms/step - loss: 18.8381 - val_loss: 8.7439

Epoch 91/200
121/121 - 0s - 2ms/step - loss: 18.7859 - val_loss: 8.7276
Epoch 92/200
121/121 - 0s - 3ms/step - loss: 18.7102 - val_loss: 8.7135
Epoch 93/200
121/121 - 0s - 2ms/step - loss: 18.8509 - val_loss: 8.7022
Epoch 94/200
121/121 - 0s - 3ms/step - loss: 18.4496 - val_loss: 8.6888
Epoch 95/200
121/121 - 0s - 2ms/step - loss: 18.6287 - val_loss: 8.6800
Epoch 96/200
121/121 - 0s - 3ms/step - loss: 18.4114 - val_loss: 8.6630
Epoch 97/200
121/121 - 0s - 2ms/step - loss: 18.7350 - val_loss: 8.6539
Epoch 98/200
121/121 - 0s - 3ms/step - loss: 18.5229 - val_loss: 8.6417
Epoch 99/200
121/121 - 0s - 3ms/step - loss: 18.1560 - val_loss: 8.6340
Epoch 100/200
121/121 - 0s - 3ms/step - loss: 18.3762 - val_loss: 8.6195
Epoch 101/200
121/121 - 0s - 2ms/step - loss: 18.4043 - val_loss: 8.6087
Epoch 102/200
121/121 - 0s - 2ms/step - loss: 18.0133 - val_loss: 8.5992
Epoch 103/200
121/121 - 0s - 2ms/step - loss: 18.4696 - val_loss: 8.5874
Epoch 104/200
121/121 - 0s - 3ms/step - loss: 18.5018 - val_loss: 8.5749
Epoch 105/200
121/121 - 0s - 3ms/step - loss: 18.1270 - val_loss: 8.5667
Epoch 106/200
121/121 - 0s - 2ms/step - loss: 18.0545 - val_loss: 8.5586
Epoch 107/200
121/121 - 0s - 2ms/step - loss: 18.1155 - val_loss: 8.5503
Epoch 108/200
121/121 - 0s - 2ms/step - loss: 17.6787 - val_loss: 8.5401
Epoch 109/200
121/121 - 0s - 2ms/step - loss: 17.8990 - val_loss: 8.5294
Epoch 110/200
121/121 - 0s - 2ms/step - loss: 17.8791 - val_loss: 8.5178
Epoch 111/200
121/121 - 0s - 2ms/step - loss: 17.7268 - val_loss: 8.5094
Epoch 112/200
121/121 - 0s - 2ms/step - loss: 17.8260 - val_loss: 8.4995
Epoch 113/200
121/121 - 0s - 2ms/step - loss: 17.5905 - val_loss: 8.4873
Epoch 114/200
121/121 - 0s - 2ms/step - loss: 17.6476 - val_loss: 8.4740
Epoch 115/200
121/121 - 0s - 2ms/step - loss: 17.5049 - val_loss: 8.4667
Epoch 116/200
121/121 - 0s - 2ms/step - loss: 17.4171 - val_loss: 8.4619
Epoch 117/200
121/121 - 0s - 3ms/step - loss: 17.4891 - val_loss: 8.4548
Epoch 118/200
121/121 - 0s - 3ms/step - loss: 17.2399 - val_loss: 8.4461
Epoch 119/200
121/121 - 0s - 2ms/step - loss: 17.7266 - val_loss: 8.4394
Epoch 120/200
121/121 - 0s - 2ms/step - loss: 17.1003 - val_loss: 8.4281

Epoch 121/200
121/121 - 0s - 2ms/step - loss: 17.1450 - val_loss: 8.4250
Epoch 122/200
121/121 - 0s - 3ms/step - loss: 17.2077 - val_loss: 8.4173
Epoch 123/200
121/121 - 0s - 2ms/step - loss: 17.1851 - val_loss: 8.4081
Epoch 124/200
121/121 - 0s - 2ms/step - loss: 17.0245 - val_loss: 8.4005
Epoch 125/200
121/121 - 0s - 2ms/step - loss: 17.0555 - val_loss: 8.3916
Epoch 126/200
121/121 - 0s - 2ms/step - loss: 17.3224 - val_loss: 8.3841
Epoch 127/200
121/121 - 0s - 2ms/step - loss: 17.0983 - val_loss: 8.3764
Epoch 128/200
121/121 - 0s - 3ms/step - loss: 17.1480 - val_loss: 8.3719
Epoch 129/200
121/121 - 0s - 2ms/step - loss: 16.9905 - val_loss: 8.3690
Epoch 130/200
121/121 - 0s - 2ms/step - loss: 17.1824 - val_loss: 8.3590
Epoch 131/200
121/121 - 0s - 3ms/step - loss: 16.8188 - val_loss: 8.3519
Epoch 132/200
121/121 - 0s - 2ms/step - loss: 16.7538 - val_loss: 8.3456
Epoch 133/200
121/121 - 0s - 2ms/step - loss: 16.7591 - val_loss: 8.3410
Epoch 134/200
121/121 - 0s - 3ms/step - loss: 16.6442 - val_loss: 8.3340
Epoch 135/200
121/121 - 0s - 2ms/step - loss: 16.8611 - val_loss: 8.3282
Epoch 136/200
121/121 - 0s - 2ms/step - loss: 16.4589 - val_loss: 8.3210
Epoch 137/200
121/121 - 0s - 2ms/step - loss: 16.5247 - val_loss: 8.3154
Epoch 138/200
121/121 - 0s - 2ms/step - loss: 16.1393 - val_loss: 8.3103
Epoch 139/200
121/121 - 0s - 2ms/step - loss: 16.7264 - val_loss: 8.3063
Epoch 140/200
121/121 - 0s - 3ms/step - loss: 16.4530 - val_loss: 8.3024
Epoch 141/200
121/121 - 0s - 2ms/step - loss: 16.5194 - val_loss: 8.2980
Epoch 142/200
121/121 - 0s - 2ms/step - loss: 16.5381 - val_loss: 8.2921
Epoch 143/200
121/121 - 0s - 2ms/step - loss: 16.4405 - val_loss: 8.2878
Epoch 144/200
121/121 - 0s - 2ms/step - loss: 16.3134 - val_loss: 8.2824
Epoch 145/200
121/121 - 0s - 3ms/step - loss: 16.5686 - val_loss: 8.2787
Epoch 146/200
121/121 - 0s - 2ms/step - loss: 16.0627 - val_loss: 8.2778
Epoch 147/200
121/121 - 0s - 2ms/step - loss: 16.2156 - val_loss: 8.2741
Epoch 148/200
121/121 - 0s - 3ms/step - loss: 16.4070 - val_loss: 8.2690
Epoch 149/200
121/121 - 0s - 3ms/step - loss: 16.3552 - val_loss: 8.2653
Epoch 150/200
121/121 - 0s - 2ms/step - loss: 16.4400 - val_loss: 8.2619

Epoch 151/200
121/121 - 0s - 2ms/step - loss: 16.3181 - val_loss: 8.2577
Epoch 152/200
121/121 - 0s - 2ms/step - loss: 15.9846 - val_loss: 8.2554
Epoch 153/200
121/121 - 0s - 2ms/step - loss: 15.9779 - val_loss: 8.2510
Epoch 154/200
121/121 - 0s - 2ms/step - loss: 15.9585 - val_loss: 8.2458
Epoch 155/200
121/121 - 0s - 2ms/step - loss: 15.8787 - val_loss: 8.2418
Epoch 156/200
121/121 - 0s - 2ms/step - loss: 15.7248 - val_loss: 8.2389
Epoch 157/200
121/121 - 0s - 2ms/step - loss: 15.9865 - val_loss: 8.2365
Epoch 158/200
121/121 - 0s - 3ms/step - loss: 15.9792 - val_loss: 8.2338
Epoch 159/200
121/121 - 0s - 2ms/step - loss: 15.9160 - val_loss: 8.2336
Epoch 160/200
121/121 - 0s - 2ms/step - loss: 15.8486 - val_loss: 8.2327
Epoch 161/200
121/121 - 0s - 3ms/step - loss: 15.9467 - val_loss: 8.2316
Epoch 162/200
121/121 - 0s - 2ms/step - loss: 15.6031 - val_loss: 8.2291
Epoch 163/200
121/121 - 0s - 2ms/step - loss: 15.5325 - val_loss: 8.2285
Epoch 164/200
121/121 - 0s - 3ms/step - loss: 15.6076 - val_loss: 8.2267
Epoch 165/200
121/121 - 0s - 2ms/step - loss: 15.5278 - val_loss: 8.2232
Epoch 166/200
121/121 - 0s - 3ms/step - loss: 15.5186 - val_loss: 8.2229
Epoch 167/200
121/121 - 0s - 2ms/step - loss: 15.5207 - val_loss: 8.2220
Epoch 168/200
121/121 - 0s - 2ms/step - loss: 15.6605 - val_loss: 8.2204
Epoch 169/200
121/121 - 0s - 2ms/step - loss: 15.6555 - val_loss: 8.2190
Epoch 170/200
121/121 - 0s - 3ms/step - loss: 15.3211 - val_loss: 8.2179
Epoch 171/200
121/121 - 0s - 2ms/step - loss: 15.5223 - val_loss: 8.2168
Epoch 172/200
121/121 - 0s - 3ms/step - loss: 15.7393 - val_loss: 8.2152
Epoch 173/200
121/121 - 0s - 2ms/step - loss: 15.5104 - val_loss: 8.2140
Epoch 174/200
121/121 - 0s - 2ms/step - loss: 15.7142 - val_loss: 8.2124
Epoch 175/200
121/121 - 0s - 2ms/step - loss: 15.3751 - val_loss: 8.2098
Epoch 176/200
121/121 - 0s - 2ms/step - loss: 15.4906 - val_loss: 8.2105
Epoch 177/200
121/121 - 0s - 3ms/step - loss: 15.3598 - val_loss: 8.2091
Epoch 178/200
121/121 - 0s - 3ms/step - loss: 15.1495 - val_loss: 8.2066
Epoch 179/200
121/121 - 0s - 2ms/step - loss: 15.3605 - val_loss: 8.2060
Epoch 180/200
121/121 - 0s - 3ms/step - loss: 15.3270 - val_loss: 8.2044

```

Epoch 181/200
121/121 - 0s - 2ms/step - loss: 15.1333 - val_loss: 8.2020
Epoch 182/200
121/121 - 0s - 2ms/step - loss: 15.4351 - val_loss: 8.2012
Epoch 183/200
121/121 - 0s - 2ms/step - loss: 15.2774 - val_loss: 8.1993
Epoch 184/200
121/121 - 0s - 3ms/step - loss: 15.1772 - val_loss: 8.1995
Epoch 185/200
121/121 - 0s - 3ms/step - loss: 15.2011 - val_loss: 8.1982
Epoch 186/200
121/121 - 0s - 3ms/step - loss: 15.3532 - val_loss: 8.1984
Epoch 187/200
121/121 - 1s - 5ms/step - loss: 15.0340 - val_loss: 8.1979
Epoch 188/200
121/121 - 0s - 2ms/step - loss: 15.0379 - val_loss: 8.1965
Epoch 189/200
121/121 - 0s - 3ms/step - loss: 15.0680 - val_loss: 8.1950
Epoch 190/200
121/121 - 0s - 2ms/step - loss: 14.9889 - val_loss: 8.1930
Epoch 191/200
121/121 - 0s - 3ms/step - loss: 15.1607 - val_loss: 8.1920
Epoch 192/200
121/121 - 0s - 2ms/step - loss: 15.0040 - val_loss: 8.1920
Epoch 193/200
121/121 - 0s - 2ms/step - loss: 14.9702 - val_loss: 8.1928
Epoch 194/200
121/121 - 0s - 3ms/step - loss: 15.0247 - val_loss: 8.1941
Epoch 195/200
121/121 - 0s - 2ms/step - loss: 14.7758 - val_loss: 8.1931
Epoch 196/200
121/121 - 0s - 2ms/step - loss: 14.8594 - val_loss: 8.1926
Epoch 197/200
121/121 - 0s - 2ms/step - loss: 15.0426 - val_loss: 8.1922
Epoch 198/200
121/121 - 0s - 3ms/step - loss: 14.9661 - val_loss: 8.1916
Epoch 199/200
121/121 - 0s - 2ms/step - loss: 14.9118 - val_loss: 8.1912
Epoch 200/200
121/121 - 0s - 3ms/step - loss: 14.7804 - val_loss: 8.1902
Restoring model weights from the end of the best epoch: 200.

```

```
Out[12]: <keras.src.callbacks.history.History at 0x7ef1a0703690>
```

This is how our 'tuned' model design will look, with a total of 2 hidden layers (with 6 and 21 units) and a final output layer:

```
In [13]: model.summary()
```

Model: "sequential_1"

Layer (type) Param #	Output Shape	
dense_5 (Dense) 55	(None, 11)	
dropout_4 (Dropout) 0	(None, 11)	
dense_6 (Dense) 252	(None, 21)	
dropout_5 (Dropout) 0	(None, 21)	
dense_7 (Dense) 22	(None, 1)	

Total params: 989 (3.87 KB)

Trainable params: 329 (1.29 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 660 (2.58 KB)

And here we will end the deep learning part of our task today! We have built a somewhat simple model using hyper-parameter tuning. Now, let's see whether the predictions (out-of-sample, of course) of this model are actually useful in a trading strategy for the test period selected.

3. Financial performance of the model

Now, we will check if the predictions delivered in the test sample are valid for developing a trading strategy!

3.1 Backtesting out-of-sample (in test sample)

As usual, we will now perform a backtesting of a trading strategy based on the predictions of the model:

```
In [14]: y_pred = model.predict(X_test)
```

101/101 ————— 0s 925us/step

```
In [15]: df_predictions = pd.DataFrame(
    {
        'Date': test_time, # No flatten needed for DatetimeIndex
        'Pred': y_pred.flatten() if y_pred.ndim > 1 else y_pred, #
        'Ret': Ret_vector.flatten() if Ret_vector.ndim > 1 else
Ret_vector, # Flatten if 2D
    }
)
df_predictions.tail()
df = df_predictions
df.tail()
```

```
Out[15]:
```

	Date	Pred	Ret
3199	2025-03-25	1.448421	1.37
3200	2025-03-26	1.211321	-0.99
3201	2025-03-27	1.418115	1.05
3202	2025-03-28	1.219985	-2.66
3203	2025-03-31	1.213160	1.94

As we have done other times before, we will compute the returns of 3 strategies in the test period:

- **Buy and hold:** simply buys the stock at the beginning of the test period and holds it until the end.
- **Long-only:** this strategy either goes long in the stock (if the predicted 25-day return is positive) or goes to cash (with a 0% return associated).
- **Long/Short** this strategy goes long (short) in the stock when the 25-day ahead return predicted by the model is positive (negative)

```
In [16]: df['Ret'] = df['Ret'] / 100

df["Positions"] = np.where(df["Pred"] > 0, 1, -1)
df["Strat_ret"] = df["Positions"].shift(1) * df["Ret"]
df["Positions_L"] = df["Positions"].shift(1)
df["Positions_L"][df["Positions_L"] == -1] = 0
df["Strat_ret_L"] = df["Positions_L"] * df["Ret"]
df["CumRet"] = df["Strat_ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)
df["CumRet_L"] = df["Strat_ret_L"].expanding().apply(lambda x: np.prod(1 + x) - 1)
df["bhRet"] = df["Ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)

Final_Return_L = np.prod(1 + df["Strat_ret_L"]) - 1
Final_Return = np.prod(1 + df["Strat_ret"]) - 1
Buy_Return = np.prod(1 + df["Ret"]) - 1

print("Strat Return Long Only =", Final_Return_L * 100, "%")
print("Strat Return =", Final_Return * 100, "%")
print("Buy and Hold Return =", Buy_Return * 100, "%")
```

/tmp/ipykernel_13387/2786902323.py:6: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df["Positions_L"][df["Positions_L"] == -1] = 0
```

Strat Return Long Only = 739.5384870082764 %

Strat Return = 407.82058285920135 %

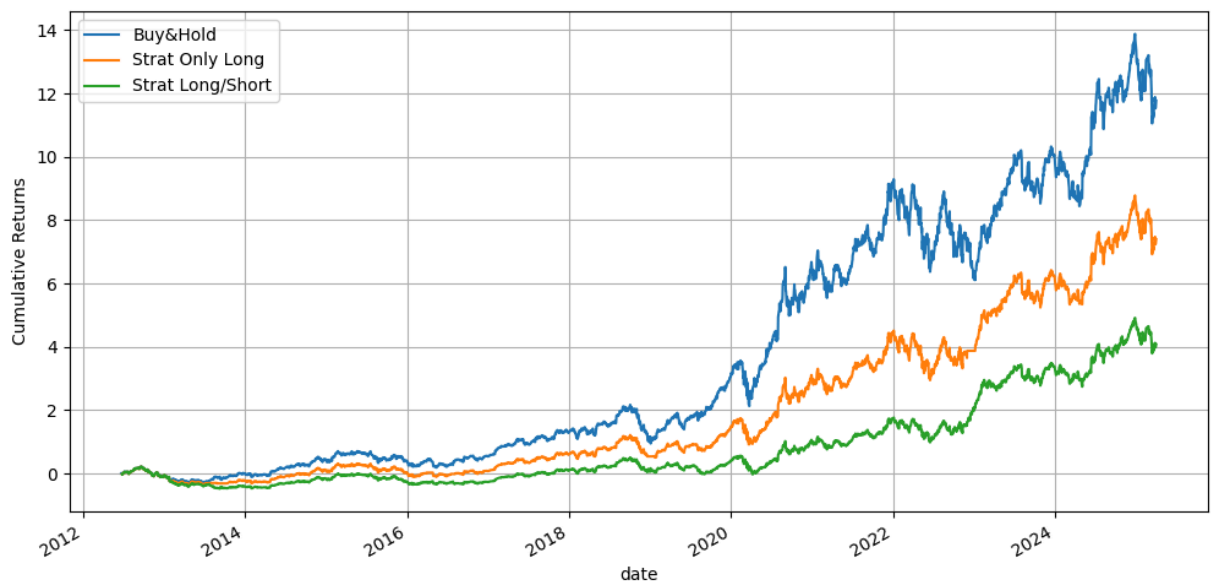
Buy and Hold Return = 1177.106632396256 %

Judging by the numbers obtained here, it seems that, under these tuned hyper-parameters and, remember, for the testing period of our choice, our model does a fairly decent job in timing the AAPL stock. Let's better see it graphically:

```
In [17]: import matplotlib.pyplot as plt

fig = plt.figure(figsize=(12, 6))
ax = plt.gca()
df.plot(x="Date", y="bhRet", label="Buy&Hold", ax=ax)
df.plot(x="Date", y="CumRet_L", label="Strat Only Long", ax=ax)
df.plot(x="Date", y="CumRet", label="Strat Long/Short", ax=ax)
plt.xlabel("date")
plt.ylabel("Cumulative Returns")
plt.grid()
plt.show()

df.describe()
```



Out[17]:

	Date	Pred	Ret	Posit ions	Strat _ret	Posit ions_ L	Strat _ret_ L	Cum Ret	Cum Ret_ L	bhRe t
coun t	3204	3204. 0000 00	3204. 0000 00	3204. 0000 00	3203. 0000 00	3203. 0000 00	3203. 0000 00	3203. 0000 00	3203. 0000 00	3204. 0000 00
mea n	2018- 11-08 16:05 :23.5 9550 5664	1.702 753	0.000 952	0.859 551	0.000 664	0.929 753	0.000 807	0.787 334	1.956 566	3.690 040
min	2012- 06-22 00:00 :00	-1.78 8622	-0.12 8600	-1.00 0000	-0.12 8600	0.000 000	-0.12 8600	-0.47 5654	-0.36 8951	-0.31 4152
25%	2015- 08-31 18:00 :00	0.736 493	-0.00 7500	1.000 000	-0.00 7700	1.000 000	-0.00 6600	-0.16 0304	0.076 339	0.429 832
50%	2018- 11-06 12:00 :00	1.479 906	0.001 000	1.000 000	0.000 700	1.000 000	0.000 000	0.158 475	0.787 262	1.579 304
75%	2022- 01-13 06:00 :00	2.439 269	0.010 300	1.000 000	0.010 000	1.000 000	0.009 000	1.293 553	3.557 529	7.280 817
max	2025- 03-31 00:00 :00	8.799 627	0.119 800	1.000 000	0.119 800	1.000 000	0.119 800	4.914 728	8.778 340	13.87 4818
std	NaN	1.450 142	0.017 669	0.511 130	0.017 684	0.255 602	0.016 857	1.347 783	2.396 120	3.955 993

4. Conclusion

In this last lesson of Module 1, we have implemented a basic tuning of model hyperparameters. Even though we have performed a somewhat simple tuning that focused on very few hyper-parameters, there are 2 things that we have likely come to realize: (i) tuning is computationally costly, and (ii) it can actually make a huge difference!. In the next module, we will continue to explore more advanced Deep Learning models, that will allow us to gradually increase the complexity of the tasks at hand and its implementation.

See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.